第二章 计算思维的核心方法与 AI 辅助

一份导览程序成功计算出“最短路线”，访客却到了不能进入的门；两个模块各自通过测试，合起来却使用不同的地图版本；一个规则在历史记录上很准确，换到新场次就频繁失效。这些问题不只要求把代码写对，还要求解释表示、接口、经验和算法之间的关系。第二章把第一章建立的问题意识推进到能够实际推演、比较和修正的方法。

本章围绕抽象、分解、模式识别、算法设计、评估与优化展开。五种方法不是必须依次经过的流水线。一个新反例可能要求回到表示，一次版本错误可能只需检查接口，一个明确非负的静态图可以直接调用已有最短路径方法。AI 参与提出候选、提取材料和构造测试；采用何种结果，要由可复核的推理与证据决定。

学习目标

· LO1：比较两种表示怎样改变可行动作与结果，用具体反例指出必须保留的信息和状态。

· LO2：按职责与接口分解任务，解释依赖、版本及语义条件怎样决定局部结果能否组成整体。

· LO3：从有限记录提出规则、保留例外并检查新样本，说明预测、结构相似和因果解释的区别。

· LO4：依据状态、代价与目标选择搜索或排序方法，追踪关键不变量、终止条件及反例，区分可行和最优。

· LO5：在相同输入条件下比较基线与候选，核对分母、分组、优化总成本和自动执行的恢复前提。

· LO6：在新的导览任务中交付模型、依赖、两种策略、测试与修订说明，核验 AI 辅助产物并公开结论边界。

学习路线

2.1 先问“当前表示还有没有足够的信息”，2.2 追问“分开的部分凭什么能接起来”，2.3 检查“观察到的规则可以用到哪里”，2.4 通过状态推演判断“怎样计算以及为什么成立”，2.5 把输出放回比较、反馈与执行条件。章末 E06 用完整的新图整合这些方法，题解统一集中在章末，建议先独立完成再对照。

默认先备为编程、基本数据结构、集合关系和简单图算法。必要符号和测量口径随例子说明，不要求已经学习统计学习、形式验证或运筹优化。核心任务均有手工或短程序路径，不依赖付费模型、特定品牌账户或真实校园数据。关于先备掌握与练习耗时的判断尚需教师和学生验证；本章原创数据是教学模拟。

本章目录

2.1 抽象：选择信息、建立表示、检查遗漏.....4

2.2 分解：组织子问题与依赖关系.....15

2.3 模式识别：归纳规则、保留例外.....26

2.4 算法设计：搜索、排序与决策过程.....37

2.5 评估、优化与自动化：建立反馈闭环.....49

章末综合训练.....61

本章回顾.....64

关键术语.....64

参考解与讨论.....65

参考文献.....77

2.1 抽象：选择信息、建立表示、检查遗漏

把一幅地图转换成图并不困难，困难在于判断转换后还能回答什么。平面距离相同的两条道路，对步行者可能可以互换，对使用轮椅的人却可能具有完全不同的可行性。熟悉图结构和最短路径程序，并不能替我们决定节点该代表建筑、入口还是某一时刻的人员状态。本节从一个导览表示的失败出发，讨论怎样使信息取舍与任务相匹配。

2.1.1 一张看起来正确却无法使用的地图

讲解示例：到达建筑还是到达入口。一个原创教学情境中，访客从西门去资料中心。设计者在地图上把每栋建筑表示为一个节点，把道路表示为边，记录步行分钟数。程序找到“西门—资料中心”的最短路线。到达后，访客发现道路接到的是北侧台阶入口，而可使用的无障碍入口在南侧，需要绕行。程序的加法正确，错误发生在更早的表示阶段：节点“资料中心”把两个不能互换的入口合并了。

先不要急于把所有建筑细节都加回地图。如果任务只是在宣传页展示建筑之间的大致位置，一个建筑一个节点可能很合适；如果任务要求到达某个可进入的门，这种表示就不足够。表示的质量取决于它所支撑的判断，不能仅凭字段数量、视觉精细程度或文件大小评价。一张像素很高的地图可能缺少入口通行信息，一张只有六个节点的关系图却可能足以回答当前问题。

我们把资料中心的两个入口记为北门 N 和南门 M。西门到 N 需要 3 分钟，西门到 M 需要 7 分钟，N 到 M 的建筑外绕行需要 4 分钟。北门有台阶，南门可无台阶进入。如果把 N 和 M 直接压成节点 B，并仅保存最短接近时间 3，就丢失了“到达

哪一个入口”的区别。对无台阶通行任务，3 分钟不是一个精度略差的估计，而是一个对应错误动作的结果。将它修成 4 分钟或 5 分钟也无济于事，因为丢掉的首先是通行语义。

抽象是为特定目的选择对象、关系与性质，并隐藏当前判断不需要的区别。Wing 讨论抽象时强调以一个对象表示多个对象、保留共同性质并隐藏无关差异[1]。这里进一步提出一个本书用于检查抽象的办法：先找出被合并的区别，再问这些区别会不会改变可行动作、评价或输出。它是任务分析方法，不是一条保证所有现实问题都能完全形式化的定理。

2.1.2 用两个情境检查信息是否丢得过多

设真实情境为 w，表示函数为 α，得到的模型为 $\alpha(w)$。如果两个情境 w_1、w_2 被映成相同表示，而所需判断不同，依靠该表示的任何确定处理过程都无法在这两个情境上同时给出正确判断。这个结论不需要复杂证明：处理过程拿到的输入一样，就没有依据区分应当不同的结果。增加算法步骤、换用更大的模型，也不能恢复输入中没有保留而又无法另行取得的信息。

例如，一条道路在情境一中有缓坡，在情境二中有台阶。如果表示只保留“长度 80 米”，两种情境得到相同输入；任务却要求判断轮椅能否通行，那么该表示无法独立完成任务。它也许能输出“需核实”，但不能把不确定性自动变成“可通行”。发现这类反例后，应当补充通行属性或缩小系统承诺，而不是把某一次猜对当作表示充分的证明。

注意，我们比较的是同一任务下的结果。对于估算道路养护面积，台阶与缓坡是否需要区分还取决于养护工艺；对于生成一张非常粗略的空间关系图，80 米也许已经足够。抽象没有脱离用途的绝对好坏。说明“这张表足够”时，应当接着说明“足够回答哪个问题、在什么条件下”。否则读者无法知道结论是否能迁移到新任务。

若任务是选路线，单一的“是否可行”也可能不够。需要保留每个候选路线的可行性及用于比较的代价。两种表示都找到了可行路线，不代表它们保留了相同的最优结果；它们可能把通行时间取成不同口径，使两条路线的排序倒转。检查抽象时可以逐层问：可行集合是否发生变化，候选间的优劣是否发生变化，最终输出中的说明是否仍然有依据。这比笼统地问“信息是否完整”更容易发现具体问题。

信息合并在类别设计中尤其常见。把所有访客归为“普通用户”，可能删除不同的通行条件；把所有道路归为“连接”，可能删除方向；把所有开放状态归为“可用”，可能删除检查时间。这里不要求为每个人建立独立模型。可以将具有相同任务条件的人划成同一类，例如当前任务中的“允许台阶”与“要求无台阶”。类别应由会改变判断的性质建立，而不是由外观相似或已有数据库字段方便与否决定。

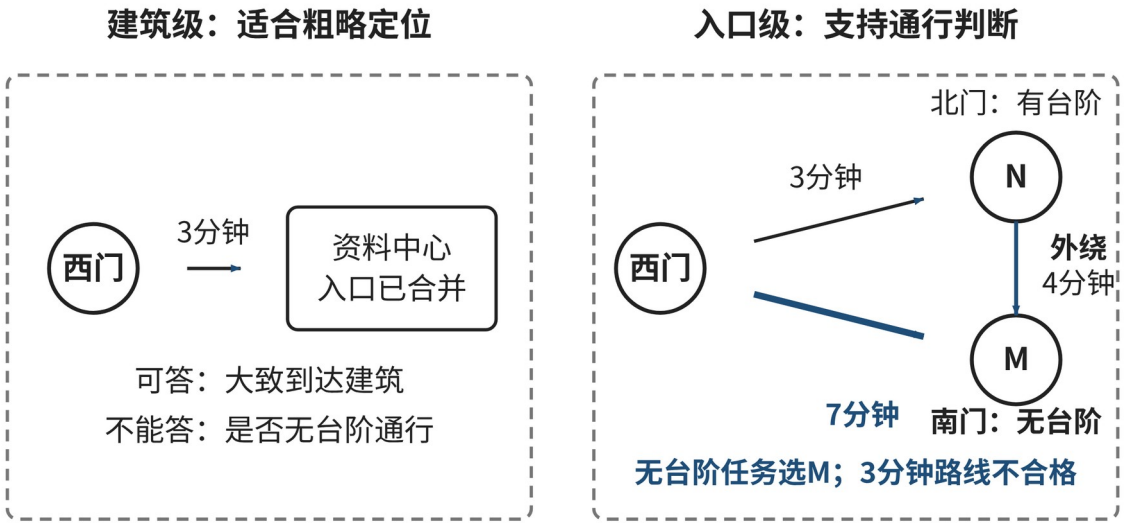


图2-1 两种导览表示及其可回答的问题

来源：本书原创教学图，依据本节入口情境绘制，时间为模拟值。

图 2-1把建筑级表示与入口级表示并置。读图时先追踪 3 分钟道路究竟接到哪一个入口，再检查将两个入口合并后，哪一项判断失去了依据。右侧的表示更详细，但它的价值在于补回当前任务需要的区别。若后来只需展示建筑所在区域，可以从右侧生成更粗的视图，不必让所有用户都阅读全部字段。

2.1.3 表示既选择对象，也选择关系

对象选对了，关系仍可能选错。以入口为节点后，若把单向通行写成无向边，程序便可能推荐一条不能反向走的路线。若把“两个入口属于同一建筑”误当成“两个入口之间零代价可通行”，程序又会制造不存在的连接。图中的边不是任何两点之间的一条线，而是当前任务所允许的一种状态变化。

进一步看合并节点的风险。假设一条路接到北门，另一条路从南门离开，北门与南门之间需要经过室外绕行。把两门合成 B 后，路径“西门—B—东门”可能偷偷包含一次从北门到南门的零时间移动。每条外部道路分别都是真的，但把它们拼起

来的内部转移是假的。抽象因此不仅要保留单个字段，还要保留组合时成立的关系。检查一条路线是否有意义，应核对相邻两条边在同一个具体端点上衔接。

方向、单位和时间基准也属于关系语义。边权写“4”，使用者不能判断它是分钟、百米还是拥堵等级。写“开放”而不说明何时核验、对谁开放，可能使上一时刻的事实变成当前承诺。适当的边记录至少说明起点、终点、代价单位、适用通行条件和证据时间。某些任务还需坡度或门宽；这些属性应由目标引入，而不是一次性把一切可能细节都录入。

数据表的主键决定对象身份。如果使用建筑简称作为入口 ID，同名入口可能被错误合并；如果同一个入口在两份表中使用不同拼写，又可能被重复建立。主键的职责不是使名称好看，而是在当前模型中稳定地区分对象。展示名称可以改变，连接关系所引用的身份必须一致。否则程序可能运行成功，却在一个被错误拼接的图上计算。

判断结果与证据状态应分开。可以用“确认可通行”“确认不可通行”“尚无足够信息”表示三种不同情况。缺失值不应自动编码为 0 分钟或不可通行，也不应为了使图连通而编码为确认可用。保守筛除未知边能够生成有证据支持的路线，但如果没有路线，只能说明在已确认子图中没有找到符合条件的路径，不能据此断言现实中绝无路线。表示越明确，结果的解释也越准确。

2.1.4 决策相同，不等于状态相同

当任务要求“先到服务点领取材料，再到出口”时，节点位置已经不足以描述全部状态。同一个人站在广场 P，可能已经领取材料，也可能尚未领取。下一步允

许做什么取决于这段经历。如果程序只保存“当前位置 P”，就会合并两个具有不同未来行动条件的状态。

可以把状态表示为“当前位置、是否已经完成领取”。到达服务点且完成领取时，第二个分量由 0 变成 1；离开其他节点不会改变它。目标状态不是任意到达出口，而是“出口、1”。这里新增的分量不是用户画像，也不是为了记录完整轨迹，它只保留未来判断所需的一项历史。一个简短的状态足以替代冗长的活动日志。

设从起点到 P 的直接路线只需 2 分钟，但不经过服务点；经过服务点再到 P 需要 5 分钟。如果搜索仅保留到 P 的最小距离 2，并删除较长路线，就可能把唯一能满足领取条件的候选一起删掉。对于只要求到 P 的任务，删除 5 分钟候选正确；对于要求领取后出站的任务，两个候选不是同一种状态，不能按同一节点距离直接比较。算法看似用了熟悉的“保留最短”规则，真正的问题是比较对象被定义错了。

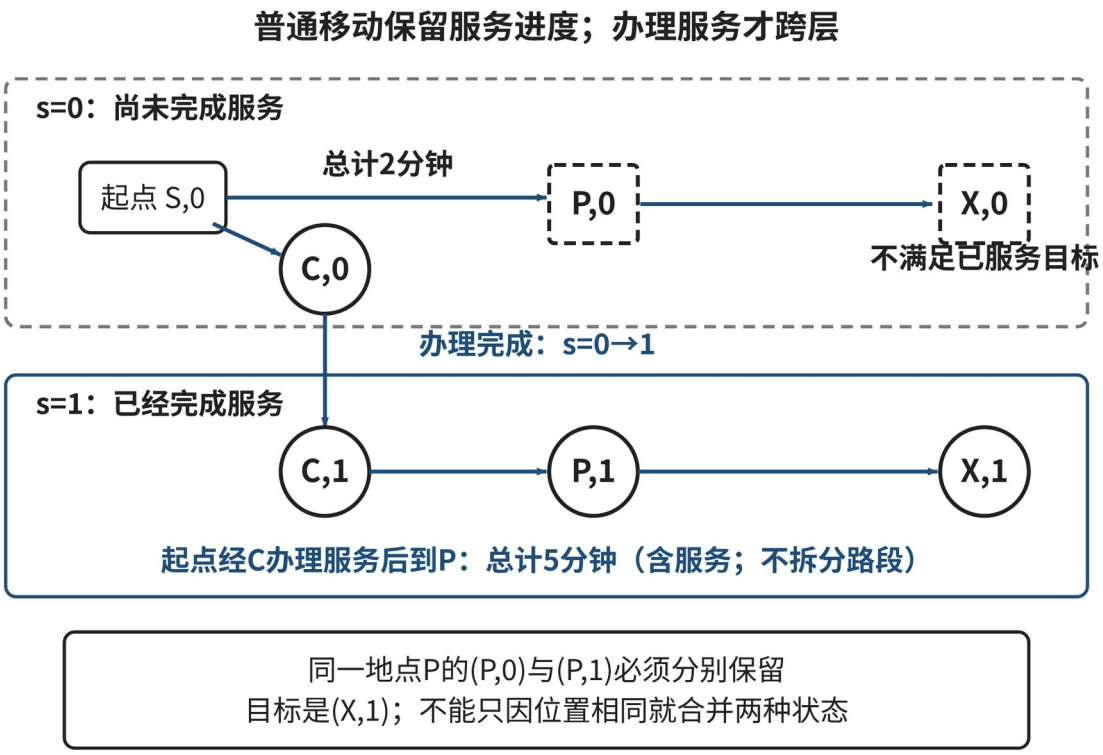


图2-2 同一位置在不同任务进度下的状态扩展

来源：本书原创教学图；0 与 1 分别表示尚未完成、已经完成指定服务，不代表现实用户记录。

图 2-2用两层节点表达这一差别。普通移动只在相同进度层中进行，完成指定服务才改变进度。阅读时应找出从哪一层到达出口才符合目标，并解释为什么“位置相同”不能作为合并全部候选的理由。2.4 讨论的算法必须在这个状态图上处理，不能把附加条件留在路径算完以后才随意检查。

时间条件也可能要求扩展状态，但不意味着任何时间问题都需要无限细的时间网格。若代价只取决于当前节点且保持静态，节点可以支撑后续计算；若一扇门只在某些时段开放，需要知道到达时刻和允许等待的规则。若一个任务还要求最多转弯两次，转弯次数或上一条边的方向可能成为状态的一部分。选择状态的原则是：被删除的过去能否改变未来合法动作或代价？若能，应保留与该影响有关的摘要。

状态扩展会增加计算量。n 个地点与一个二值任务标记组合，最多形成 2n 个状态；保留原标记后若再加入三个独立二值条件，最多会扩展到 16n 个组合。这是全部取值组合的上界，不表示每种组合都可达。设计者应利用条件关系排除无意义状态，而不能仅因为状态多就把必要条件删掉。模型大小是需要管理的成本，模型语义是不能随意牺牲的正确性条件。

2.1.5 简化、精度与保守性的取舍

并非所有任务都需要精确到秒。若目的只是给访客估计大致步行时间，分钟级表示可能足够；若必须赶上某项服务的结束时刻，取整方式就会影响可行性。把 1.8 分钟四舍五入为 2 分钟，与把 2.4 分钟四舍五入为 2 分钟，都会得到整数 2，却具有不同的截止风险。

一种办法是保留时间区间。例如一条边通常需要 3—5 分钟。若任务必须保证 10 分钟内到达，可以按上界检验；若上界总和不超过 10，且区间确实覆盖当前条件，就能得到保守的可行判断。若只按下界得到 9 分钟，则只能说存在一个乐观估计，不足以保证按时。区间上界超过 10 也不证明一定迟到，它说明当前估计不能提供所要求的保证。

上下界求和还需要注意解释。把每条边的最坏时间相加是一种保守上界，可能非常宽松；若这些极端情况互不可能同时发生，不能声称这个上界是实际最坏时间。对于一条固定路线，把各路段耗时的期望相加，得到总耗时的期望，并不要求路段耗时彼此独立；但仅凭这些期望，不能推得某个保证概率。这里区分的是不同

问题：平均体验、确定上界和概率承诺需要不同的信息。本节只要求你发现表示中的承诺变化，不要求已学习完整概率模型。

相同原则适用于金额、容量和测量误差。把若干不同金额归为一个“低费用”类别，可能足以筛选大致预算，却不能再声称比较了精确总费用。一个类别阈值附近的候选需要特别检查，因为微小的测量变化就会改变分类。抽象应同时说明被保留的分辨率和因此不能回答的问题。给出这种边界，比附上很多没有用途的小数更有价值。

2.1.6 AI 生成的表示怎样接受检查

AI 可以从一段道路说明中提取候选字段，也可以建议把哪些地点合并。它有助于发现未想到的表示方式，但生成的字段并不自动获得真实值。例如输入“资料中心南侧有坡道，开放时间另见公告”，合理输出应保留“南侧”“坡道”和“开放时间未取得”，而不能填成“全天开放”。关键不在于输出是不是 JSON，而在于它是否把未知内容改写成了确定事实。

可以给 AI 一个有限任务：仅根据提供的四条说明提出节点表和边表；每个值标注来自哪条说明；没有依据的字段留为待确认；另外列出会改变通行判断的潜在遗漏。学生随后逐条对照原材料，检查对象是否混淆、方向是否反转、单位是否补造。手工完成同样的提取与对照，可以获得相同的学习成果，不需要特定账户。

对更难的表示方案，可要求 AI 给出一对反例情境：它们在当前表示中相同，但正确判断应不同。人随后检查该反例是否符合原任务。如果反例只能通过引入题设禁止的情形成立，就没有反驳当前模型；如果它使用了允许的情形并改变结果，就

应当修订表示。提出反例与检验反例是两个不同任务，让同一个生成过程直接宣布自己的方案充分，容易遗漏共同假设中的错误。

另一个有效任务是审查一个故意不完整的候选。比如候选表只有“建筑名、距离、推荐顺序”，学生应指出它可以支持粗略游览排序，却缺少入口、方向和通行条件。要求 AI“检查模型”过于宽泛；要求它“寻找把不同入口合并后形成虚假连接的具体路径”则有明确输出和可核查对象。提示词的精细程度在这里服务于思考，不能替代材料本身。

2.1.7 E01：决定哪些信息必须保留

下面是另一份完整的原创教学地图，所有边均为有向边，分钟为确定的教学代价，其余边不存在。H 是入口，J 是台阶平台，K 是坡道平台，L 是服务大厅。任务是在 09:57 从 H 出发，使用无台阶路线到达 L，最迟 10:05 到达，不允许跳过边或把两点直接视为相通。

表2-1 E01 有向道路与通行条件

起点→终点	时间／分钟	无台阶通行	时间条件
H→J	2	否	无额外限制
J→L	2	否	必须在 10:00 之前或恰于 10:00 完成整段通行
H→K	3	是	无额外限制
K→L	4	是	无额外限制
L→H	5	是	无额外限制

来源：本书原创教学数据，不是校园道路或开放时间记录。

某候选表示把 J 和 K 合并为“中间平台 P”，只保留每对节点间最短时间；另一候选保留各节点、方向、时间、通行与时间条件。请完成以下任务：比较两种表示能否支撑当前目标；给出一条符合要求的路线并算到达时刻；为粗表示构造两个不能互换的具体路径，说明信息损失；解释为何 L 到 H 存在 5 分钟边，并不表示 H 到 L 存在同样的直接边。

再处理两个独立变化。变化一仅取消无台阶限制，截止时刻仍为 10:05，道路时间条件不变，重新比较候选。变化二恢复无台阶限制，但 K→L 的通行状态变成“尚未核实”，其余已知条件不变，说明现在可以承诺什么、还需取得什么证据。不要把“缺少已确认路径”写成“现实绝对无路”。

交付一页模型比较、具体时间推演和证据缺口说明。评价重点是被合并信息怎样改变决策、路径每一步是否有依据、变化后结论强度是否正确；不按字段数量评分。完整参考解在章末 E01，请先独立判断。若使用 AI，只让它提供候选表示或反例，并保留逐项复核，不能让它自行补齐道路信息。

2.1.8 从模型返回问题

经过本节，抽象不再只是“删除细节”。你应当能够追问一个更精确的问题：我们把哪些情境当作相同，而这种相同对所需判断是否成立？反例把抽象质量变成可讨论、可修改的对象。它也说明一种常见的修复顺序：发现错误结果后，先检查输入所区分的对象与状态，再决定是否更换算法。

反思自己的 E01 模型：哪一个区别最初看起来只是描述细节，后来却改变可行性？如果需要支持到达后返回、领取材料或不确定通行时间，原状态还够不够？你

愿意用怎样的范围说明告诉读者当前模型能够回答什么？这些回答将成为 2.2 中接口约定的内容：一个模块不能向后续步骤交付它没有保留的信息。

2.2 分解：组织子问题与依赖关系

一个场馆讲解预约系统看起来可以按顺序拆成几步：接收申请、检查资格、寻找空闲时段、登记预约、发送通知。这样的流程图能说明“先做什么、后做什么”，却还不能回答一个更难的问题：每一步由不同的人或程序实现后，怎样确认它们对“合格申请”“空闲时段”和“已预约”说的是同一件事？如果资格规定改了，日历使用的时区与申请表不同，或者两位团体负责人同时看到同一个时段为空，局部程序都可能运行成功，最后得到的却是错误结果。

分解的目标不是把一个任务切成尽可能多的小块，而是让复杂性能够被分别理解，同时让各部分在明确条件下重新组成整体。对计算机专业学生来说，真正需要掌握的是两种判断：一是子问题是否有清楚而稳定的责任边界，二是每个边界交出的结果是否满足下一部分的语义和整体不变量。函数能被单独调用，或流程图上存在一条箭头，都还不足以证明这两件事。

2.2.1 顺序步骤与模块边界不是一回事

先看预约系统的一种朴素拆法：把“收到申请”作为第一步，“查资格”作为第二步，“查日历”作为第三步，“保存并通知”作为后续步骤。它按时间顺序描述一个申请从进入系统到结束的经历，适合解释用户会看到什么，也便于定位哪一步没有执行。但它可能把同一项规则散落在多个步骤中。例如，“夜间讲解只能由已核实成年身份的团体负责人预约”若既写在资格检查，又写在可预约服务列表的

过滤条件里，身份规则更新时两处必须同时改变；只更新一处，就会出现页面显示可以申请、提交时却被拒绝的情况。

另一种拆法按可能变化的设计决定组织模块：资格策略负责解释谁可以申请哪些讲解服务；资源日历负责解释某一时段是否可用；预约记录负责把一次确认变成具有排他性的占用；通知模块负责把已经确定的状态传达给申请者。申请的业务流程仍有先后顺序，但这些模块内部可以分别维护自己的实现。资格规则随场馆规定变化时，日历内部的存储方式不必跟着变化；短信服务改成场馆应用消息时，预约记录也不必改写。

这两种拆法回答不同问题。流程分解把一段工作切成阶段，帮助安排执行顺序；模块分解关注哪些知识或设计决定应被封装，让其他部分不必依赖其内部细节。Parnas 讨论模块划分时强调，好的边界应隐藏易变的设计决定，而不是仅按程序执行的先后切段。这个思想并不意味着“所有细节都藏起来”，而是要求调用方只依赖完成任务所需的承诺；模块内部的表示和算法可以变化，只要承诺继续成立。[2]

“可能变化”也不能凭空猜测成无限多的未来需求。实际划分时，可观察当前职责中是否混合了不同原因的变化：规则改动、数据存储改动、外部服务改动是否迫使同一组代码一起修改？如果答案经常是肯定的，现有边界可能把相互独立的决定绑在了一起。相反，若为了每个字段都新建模块，调用和协调成本会大于隐藏细节带来的收益。分解的粒度要与任务的规模、变化方式和团队协作成本相称。

因此，不要把“模块多”当成分解质量的指标。一个边界值得保留，是因为它形成了可理解的职责，并减少了跨边界的隐含依赖；如果拆分后仍需同时改动所有

模块、逐个同步内部状态，或者为了完成一个简单判断必须穿过许多层，拆分可能只是把复杂性改了位置。

2.2.2 接口不仅是字段清单，还要约定含义

两个模块要合作，通常需要交换输入和输出。但接口不能只列“申请人、讲解员编号、开始时间、结束时间”四个字段。开始时间采用哪个时区？结束时刻算不算占用区间的一部分？申请人身份来自已经核实的访客账号，还是由表单自由填写？空值表示“不知道”、不适用，还是确认未通过身份核验？若调用方和提供方对这些问题理解不同，字段名完全相同也会产生语义冲突。

可以把接口看成一组对输入、输出和状态变化的承诺。对于一次预约请求，模块至少要约定：前置条件是什么；成功时输出代表什么状态；失败时区分哪些原因；调用会不会改变日历；重复调用是否可能产生两笔记录；结果依据的规则或日历版本是什么。前置条件描述“调用前必须成立的事实”，后置条件描述“返回后能够保证的结果”。这不是要求每个小函数都写形式化证明，而是让团队能在实现前后检查一份可讨论的约定。

例如，资源日历返回“空闲”，并不自然等于预约已经成立。它可能只表示在查询瞬间没有已登记的占用。若检查与登记之间允许其他请求并发进入，两个请求都可能看到空闲，再各自写入成功。此时局部查询正确，局部写入也可能正确，整体仍违反“一名讲解员同一时段最多有一组有效预约”的不变量。系统必须把检查与确认放进一个能够保证排他性的操作，或在写入时再次验证并明确处理冲突。这

个例子说明，局部结果能否组成整体，不仅取决于数据格式，还取决于执行期间状态是否会改变。

接口还需要表达“暂时无法判断”。如果身份核验数据未同步，把缺失值解释为“未核实成年身份”可能误拒申请，把它解释为“已经通过核验”则可能放行不合格申请。稳健的边界应区分“通过”“未通过”和“待核实”等结果，并说明下一步该由谁处理。明确不确定状态，比把它伪装成布尔值更容易组合，也更容易审计。

版本是语义的一部分。版本为 R7 的地点清单、按照 R8 重新划分的区域规则，可能分别有效，却不一定能够组合。让双方都提供 map_version 和 profile 等字段只是第一步，组合时还必须检查这些值满足兼容关系。相同版本标签也未必自动表示含义兼容：如果一方把 profile 理解为使用者类别，另一方把它理解为讲解服务类别，字段和取值甚至都可能相同，约定仍然不一致。要让版本可用，必须先明确由谁发布、何时变化、下游何时失效，以及兼容关系如何判定。

2.2.3 依赖图表示“需要什么”，不是所有执行细节

当多个模块通过数据和承诺连接时，可以用有向图表达依赖。若模块 B 只有在模块 A 提供某个结果后才能开始，就用边 A→B 表示“B 依赖 A”。在有向无环图中，任何按边方向都不违背依赖的节点顺序，都是一种合法拓扑序；这表示局部先后约束，不表示只有这一种执行次序。彼此没有路径连接的节点，在不考虑额外资源争用时可以并行。

图 2-3展示了这种阅读方式。先看节点标签识别每个处理环节，再沿箭头确认谁需要谁的结果，最后查看接口旁的版本标记：图上的箭头说明数据依赖，接口标签说明组合条件，二者不能互相替代。若两个输入必须引用同一份地图版本，这项要求应出现在接口契约中；单凭图上 B 和 C 都连接到 D，无法看出它们交出的信息是否兼容。图 2-3的图像由本章作者按教学模拟数据绘制。

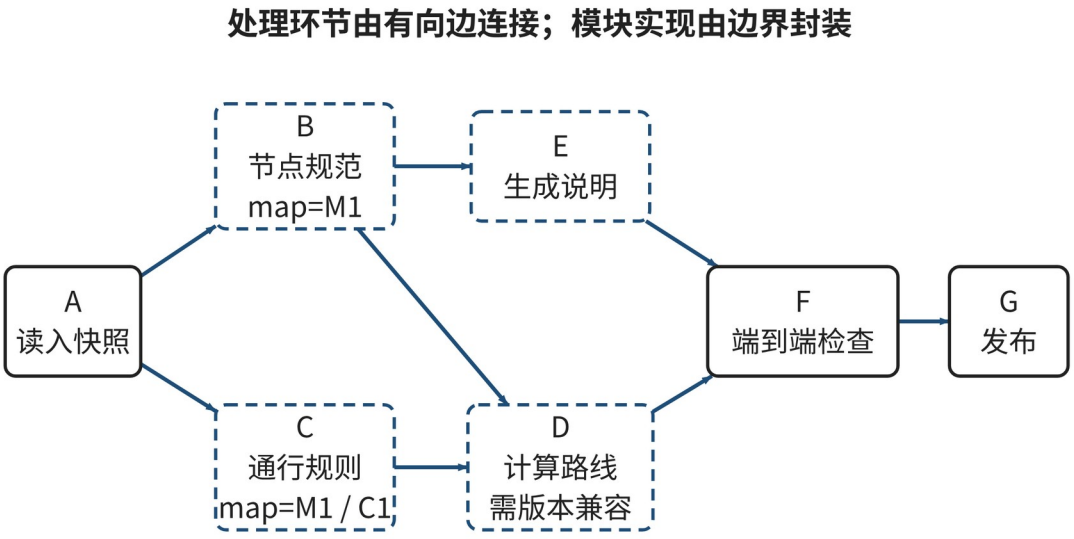


图2-3 有向无环依赖图与带版本语义的接口

来源：作者依据本节教学模拟情境绘制。

在依赖关系和用时已知、没有资源争用、各环节不可拆分且必须等待全部前置完成的简化模型中，还可以计算工期。按任意合法拓扑序扫描节点：最早开始时刻取所有前置环节完成时刻的最大值，无前置的环节从 0 开始；最早完成时刻再加上自身用时。汇合处取最大值，是因为较短分支完成后要等较长分支，而不是让两条已经并行的分支再串行执行。

例如 W 读取说明用 1 分钟；随后 Y 生成目录用 2 分钟、Z 核对署名用 4 分钟，可以同时开始；最后 V 合并两项结果用 1 分钟。Y 在第 3 分钟完成，Z 在第 5 分钟完成，V 必须等到第 5 分钟才开始，第 6 分钟结束。W、Y、Z、V 和 W、Z、Y、V 都是合法拓扑序，却不是要求 Y 和 Z 依次串行运行的时间表。沿依赖边相加，W—Z—V 共 6 分钟，控制整体完工；本节把达到最大工期的最长依赖链称为关键链。若 Y 也要 4 分钟，两条分支便同时达到 6 分钟，缩短其中一条仍可能被另一条限制。此处未包含缓冲、资源调度等管理方法；增加人员或设备限制后，单凭最长依赖链不能给出实际工期。

依赖图通常不表示所有控制条件、循环和失败恢复。如果模块 B 只在条件 C 成立时调用，单一箭头会遗漏条件；可在边或节点上补充条件。若某阶段发现数据错误后需要返回前一步修订，系统中存在反馈环，不能继续把全程当作一个简单 DAG 来计算顺序。可以把“一次提交”与“返工修订”分成两个层次描述，但必须保留回路何时发生、何时结束的规则。把循环画成无环图而不说明边界，会制造虚假的确定性。

表 2-2把“流程阶段”和“模块边界”的观察重点放在一起。读表时比较各列所隐藏的决定及对外承诺，判断某个拆分是否真正减少了对内部细节的依赖。

表2-2 流程阶段与模块边界的不同观察角度

观察对象	预约流程阶段	资格策略模块	资源日历模块	预约记录模块
主要回答	用户请求当前走到哪一步	谁可申请何种资源	哪一时段符合可用条件	申请是否已成为确认占用
易变决定	阶段顺序、交接时机	身份、年龄与权限规则	时段表示、开放日历	并发冲突与状态持久化
对外承诺	交出下一阶段可处理的状态	返回通过、拒绝或待核实	返回带版本的可用性信息	原子确认或明确冲突
需要检查	后续是否收到正确阶段结果	规则变化是否只改此处	时间边界和版本是否一致	是否保持唯一占用的不变量

来源：本书原创教学材料与作者推演；模拟数值的条件见对应题设。

依赖关系和组织边界也不应混为一谈。两个模块可能共享一个上游数据，但彼此不应互相调用；一个模块也可能在不同执行阶段被多次使用。把流程节点全都变成模块，会产生过细的管理负担；把所有处理放进一个大模块，又会让本来独立变化的决定互相牵连。判断边界时同时看数据依赖、变化原因和责任归属。

2.2.4 子问题局部正确，整体还需要组合条件

设整体任务由若干子任务组成。对子任务分别求解，只能说明每个局部处理满足自己看到的条件。要证明组合后可用，还要确认至少三类关系：其一，前一个模块提供的信息满足后一个模块的前置条件；其二，各模块没有对同一资源或状态作出相互冲突的承诺；其三，所有局部结果共同满足整体目标中的关键不变量。

可以把每个接口概括成“在假设 A 成立时，保证 G”。上游若声称提供某个结果，下游应确认自己使用它时依赖的假设确实成立；如果上游只能保证一个较弱条件，下游就不能悄悄把它当成更强保证。比如，日历只保证“查询时刻看起来可

用”，预约记录却假设“写入时仍然可用”，这两者之间缺少并发变化的处理约定。系统需要增加一次原子检查，或让记录模块拒绝过期结果并让上游重新决策。

整体检查也会暴露资源耦合。设计上相互独立的两个子任务，运行时可能共享同一数据库、限额或工作人员。依赖图没有箭头，不表示现实中永远互不影响；它只表示当前契约没有写出先后依赖。若共享资源会改变完成条件，就要补充容量或互斥约束，重新判断哪些任务可以并行。反过来，若实验设定明确规定资源无限且输出完整后才可用，可以在本次推演中忽略这些冲突，但应把假设写出来，避免把简化模型误当作实际运行结论。

还有一种容易漏掉的组合条件是“同一时点”。上游可能给出一份有效的资格结果，下游也可能给出一个有效的可用时段，但如果二者依据的账户状态、政策版本或日历更新时间不同，组合后未必对应同一个现实状态。判断模块输出能否拼接时，应同时核对对象身份、适用时间和依据版本。若其中一项无法确认，应保留“需要复核”的结果，不能仅凭每个局部值看起来合理就自动产生最终承诺。

模块输出还可能带有质量等级、时间范围和证据来源。下游不能只问“字段是否存在”，还要问“值在什么条件下可信”。例如，资格服务返回的状态来自多久以前？失效后哪些预约需要重新确认？如果不同模块分别缓存同一规则，它们需要共享失效机制，或由单一权威模块解释规则。否则每个模块在自己的局部测试里通过，集成时仍会因读取时点不同而产生矛盾。

2.2.5 选择边界并安排集成验证

可以按三步检查一份分解方案。第一，说明每个子问题的责任和它不负责的事情；“处理预约”过于宽泛，“只负责在确认时段内创建唯一有效记录”更容易检验。第二，列出每个接口的语义，包括单位、时间区间、标识符、版本、错误状态和副作用；对决定是否可组合的字段，明确谁是权威来源。第三，把整体不变量重新写回组合过程，例如“同一讲解员的有效时段不能重叠”“只有已确认的预约才能触发成功通知”，逐项检查所有路径是否满足。

集成测试应专门安排边界情形，而不只是正常输入。可以测试：字段缺失时是否进入待核实；旧版本结果是否拒绝；时段恰好首尾相接时是否重叠；两个请求同时提交时是否只产生一个确认；通知失败时预约状态是否仍然真实。每个测试对应一条接口承诺或整体不变量，失败时能定位是哪项约定未成立。用“全部模块各自单测通过”代替集成检查，会漏掉跨模块语义不一致。

契约测试适合验证边界两端对同一输入是否作出约定的解释，整体性质测试则检查多个边界连接后的结果。比如，单测可以确认日历把结束时刻作为不占用的右端点；跨模块测试还要确认预约记录也采用同一约定，否则连续相邻的两个时段仍可能被一方判为重叠。测试名称应指出它覆盖哪项承诺，输入应包含恰在边界上的例子。这样发生回归时，团队能分清问题来自模块内部实现，还是接口约定没有被一致执行。

边界越多，协调和沟通也越多。接口不该成为写完后才补的一层文档，而应是拆分工作时共同商定的语言。早期可以用小型样例和反例试接口：提供一组边界数

据，分别让两个模块解释其含义，再比较是否得到兼容结果。若人必须通过私下口头约定补充字段含义，那么真正的接口还没有写出来。

AI 可以帮助列出候选模块、草拟接口字段或从需求文档中抽取可能的约束，但生成的划分只是待验证的假设。给它一段需求并要求“拆成模块”时，结果经常会把流程标题改写成模块名，却没有说明谁拥有规则、输出何时有效、版本如何兼容。更有用的请求是要求它同时列出每个边界隐藏的决定、输入含义、前置条件、承诺、错误状态、可能的并发冲突和一个反例；随后由团队对照真实需求和代码逐项确认。

还可以让 AI 扮演下游调用方，尝试只用接口文档完成调用：它若不得不猜测时区、空值含义或状态来源，说明接口文本仍有歧义。接着让它生成边界测试候选，再由人确认测试是否覆盖真实不变量并实际执行。AI 能加速提出问题，不能替团队决定权限规则、资源权威来源或失败后果；这些决定要由对系统负责的人依据需求、数据和组织约束作出。

2.2.6 练习 E02：让局部结果接成完整交付

以下为原创教学模拟数据。假设处理环节之间没有资源冲突，所有环节可以无限并行；启动时刻为 0。每个环节用时确定，输出只有在环节完成后才能使用。

表2-3 E02 交付环节与依赖

环节	工作内容	用时（分钟）	前置环节
A	读取快照	2	无
B	统一节点标识	4	A
C	核对通行规则	3	A

环节	工作内容	用时（分钟）	前置环节
D	计算路线	5	B、C
E	生成节点说明	2	B
F	端到端检查	3	D、E
G	发布交付包	1	F

来源：本书原创教学材料与作者推演；模拟数值的条件见对应题设。

路线计算的输入来自节点与规则两份资料；路线以及规则摘要都要携带 map_version 和 profile。两份资料不能只因为字段名称一致就被当作来自同一版本。

请提交一张依赖图，写出两种不同的合法执行序列，并计算无限并行条件下的最早完成时刻及一条或多条关键依赖链。然后诊断这样的接口交接：B 输出 R7 节点，C 输出 R8 规则。说明 D 能否安全地将其作为同一快照直接组合，并指出核验接口还应比较什么。最后把 D 的用时从 5 分钟改为 2 分钟，重算最早完成时刻与关键依赖链；说明哪些分支的完成时间现在会影响 F 的启动。

成果包括依赖图、两条序列、时间推演表和接口诊断。评价时检查边是否符合题意、序列是否满足全部依赖、开始时刻是否等于所有前置环节完成时刻的最大值、关键链是否达到项目完工时刻，以及版本结论是否依据语义兼容条件。若图画对了但把“字段同名”当作“版本一致”，接口诊断仍不完整。

参考推演见章末“E02参考解与讨论”；先写出各环节的开始依据，再核对完成时刻。

【本节反思】如果每个子模块都能独立通过自己的测试，整体错误最可能藏在哪些接口假设中？对其中一条假设，补写一个能暴露问题的边界测试，并说明它检查的是局部承诺还是整体不变量。

2.3 模式识别：归纳规则、保留例外

一份测试日志里出现许多相似报错，或者一组观察数据里反复出现某些共同特征，很容易让人相信自己“看到了规律”。难处在于：相似究竟指向一个可以复用的结构，还是样本偶然排在一起？一个规则在已见记录上表现很好，是因为它抓住了稳定关系，还是因为它记住了这几条记录？模式识别的价值不是替重复现象命名，而是从有限观察提出可检验的规则，并明确哪些条件下可以使用、哪些反例会使它失效。

本节的核心能力，是从样本构造有限规则，同时保留例外、时间边界和不确定性。初学者容易把“常一起出现”当作“可以互相解释”，把“开发数据上全对”当作“未来也可靠”。以下先看一种不依赖数值阈值的结构模式，再讨论怎样评价预测规则，避免把模式识别缩减成调一个分类器。

2.3.1 相似记录背后可能有不同的模式

“模式”至少有几种常见含义。第一种是重复的取值组合，例如某类输入经常对应某类结果；第二种是事件之间重复的顺序或关系，例如一个状态总在另一个状态之前出现；第三种是多个表面不同的问题具有相同的结构，例如不同程序都发生了“旧状态覆盖新状态”。这些模式可以帮助我们压缩描述、提出规则或迁移已有处理方式，但它们所支持的判断不同。

归纳前还要确定观察单位。同一段运行日志可以按“事件”“任务”或“整次运行”统计；一位用户可以对应许多操作记录；一个场次也可能包含多名观察者。单位不同，频率和例外的含义会变。如果把同一任务的十条日志当成十个独立样本，表面上证据变多了，实际可能只是重复看见同一次事件。定义单位时要写出“一行代表什么、重复记录如何处理、缺失记录是否意味着没有事件”。否则，后来比较的不是同一种对象。

观察到输入 X 和结果 Y 经常同时出现，只能说明当前资料里存在一种关联。它可能适合用于预测，也可能只是二者共同受第三个条件影响；如果要解释因果关系，还需要额外的比较设计、干预或领域证据。本章第三节只训练如何从观察归纳有限规则并检验其预测表现，不把预测分数当作原因解释。学生后续学习批判性思维与因果论证时，才系统分析这些证据要求。

结构模式也不等于“所有记录一模一样”。假设一组原创教学模拟的文件同步轨迹中，两次失败都表现为：客户端甲读取版本 v0，客户端乙也读取 v0；甲基于 v0 提交修改并生成 v1；乙随后仍基于 v0 提交，覆盖了甲的新内容。另一次相似轨迹中，乙在甲提交后重新读取 v1，再提交自己的修改，没有覆盖。表面上三条轨迹都有两个客户端写入，但真正有解释价值的模式是：两个修改是否都建立在同一个旧版本上，以及后写入者是否检查了当前版本。

如果只统计“两个客户端写入”这一个特征，成功轨迹也会被误报；如果把甲、乙的名称和时间戳逐字记住，规则又过于具体，无法迁移到其他客户端。较好的表示保留决定结果的关系：并发读取同一版本、先后提交、缺少版本冲突检查。它允许用户身份、文件名和具体时间变化，但仍能识别相同的失效结构。

然而，轨迹中存在这种顺序仍不足以证明它必然导致所有覆盖问题。也可能有服务端合并策略或重试机制介入。可以提出一个更窄的假设：“在没有冲突检测且后写入直接替换当前内容时，基于旧版本的提交可能覆盖先前修改。”随后构造边界测试：让两个客户端基于同一版本并发修改，观察是否拒绝冲突、合并还是覆盖；再令后写者先重读最新版本，比较结果。只有将模式转成可预测的操作并检验反例，归纳才从观察走向可用知识。

2.3.2 从样本到规则，要说清适用范围

当目标是预测一个尚未观察到的结果时，可把任务表示为：在预测时刻 t，使用当时可获得的输入 x，给出未来或未知结果 y 的候选估计。这里的“当时可获得”很关键。若某个变量在 y 已经发生之后才产生，它即使与结果高度相关，也不适合作为预测输入。若任务改成事后诊断，同一字段可能有用；任务定义变化，合法信息边界也随之变化。

一个可解释的规则往往由“适用对象、输入条件、输出结论”组成。例如，条件组合规则可能要求数值达到阈值，并且某个事前已知类别属于指定范围。规则的表达要完整说明：阈值适用于什么单位；缺失值怎样处理；两个条件同时满足还是满足其一；规则覆盖哪些对象和时间；输出表示风险、分类还是行动建议。遗漏这些定义，规则无法复现，团队也无法判断例外来自数据错误、条件边界还是概念本身。

规则越具体，越容易贴合已见样本，却越可能记住偶然细节；规则越简单，越容易解释和迁移，却可能忽略真正重要的差异。比如，只按总量设定一个界线，可

能把高容量系统与低容量系统放在一起；增加系统类别作为条件也许改善表达，但每增加一条特例都会带来新的数据需求和测试负担。简洁不是目的本身，复杂也不天然意味着准确；要看新增条件是否能在未参与设计的记录中重复支持原判断。

比较规则复杂度时，可以问：“去掉这条条件，哪些具体判断会改变？改变是否有足够证据支持？”如果条件只让开发集中的一条记录被正确分类，却没有更多同类观察，新增它可能只是记住特例；若它代表一个稳定且可事先观测的群体差异，则把它删掉也可能掩盖重要机制。两者都需要由后续样本区分。规则说明应留下反例，不因它降低分数就自动删除，也不因为它让故事更完整就任意添加解释。

适用范围可以画成一个区域：区域内有足够观察支持当前规则，边缘处有相邻但尚不确定的样本，区域外则不应直接外推。读图 2-4时，先找规则的输入条件，再区分已有支持、边界样本和未观测区域；不要把边界线误读成自然界中已被证明的分界。图中区域只是帮助讨论模型适用范围的教学示意，没有真实观测点或统计置信区间。

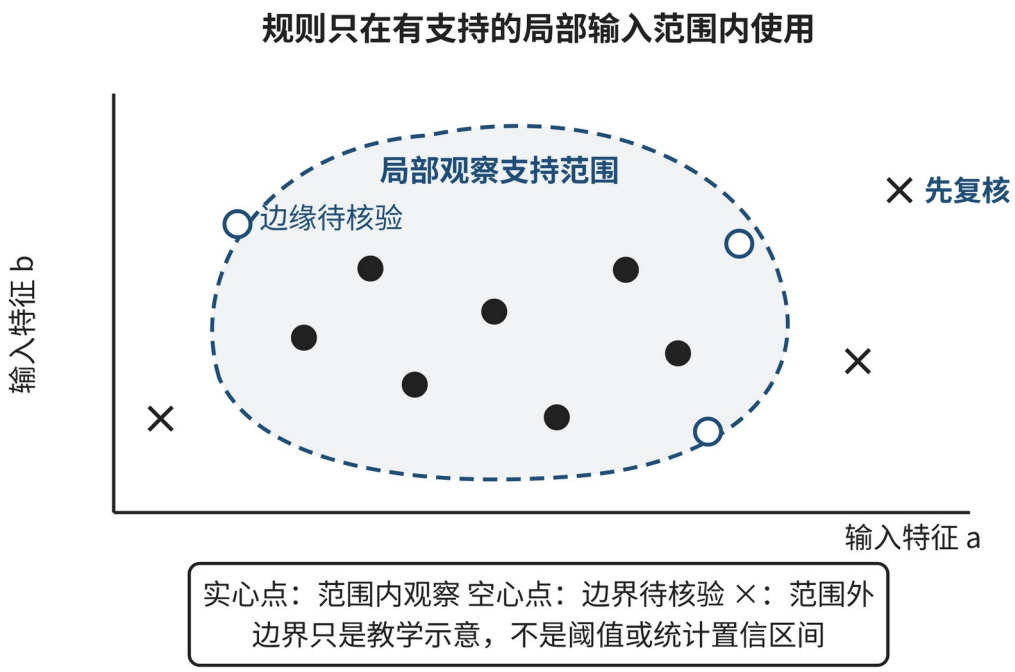


图2-4 观察样本、规则适用区域与例外边界

来源：作者按概念关系绘制的教学示意图，非实测数据。

图 2-4中的样本点用于提醒：几条相邻记录可能支持一项局部规则，但空白区域里是否仍然成立尚未得到观察。若新样本落在既有边界之外，应触发复核，而不是让规则自动扩展；若例外反复出现，也可能说明原表示遗漏了一个重要条件。图示本身不提供阈值，也不替代数据检验。

2.3.3 例外不只是噪声，可能指出模型少了什么

规则遇到反例时，不应先急着删去反例。需要先区分几种情况：记录或标签有误；观测过程改变，导致数据含义不再一致；输入字段遗漏了一个影响结果的条件；规则本身只在更窄范围内成立；或者结果确实包含不可避免的随机变化。每一种情况都要求不同的修正，不能把所有不合预期的记录都称为“噪声”。

可以用一张规则卡片记录：目标、可用输入及其采集时刻、输出含义、适用对象、规则条件、支持样本、反例、数据版本和下一次复核时间。表 2-4比较了几种规则表达的能力与风险。选择表示时要问，它保留了什么结构、略去了什么差异、怎样暴露不适用情况，而不只看它是否容易写进代码。

表2-4 模式的不同表示及其边界

表示方式	保留的结构	适合提出的问题	常见风险
单一阈值	一个数值与结果的界线	结果是否随某变量单调变化	忽略群体容量或测量口径差异
条件组合	两个以上条件的交互	某类别下是否需要不同界线	条件越加越多，样本支持变稀
顺序或关系	事件先后、状态连接	失效是否由某种操作结构触发	把共现顺序误当作因果机制
例外清单	已知不适用情形	规则在哪些边界必须暂停	例外不断增长，暴露规则过窄

来源：本书原创教学材料与作者推演；模拟数值的条件见对应题设。

规则还要绑定它所依赖的数据定义。若计数方式、标签标准或对象边界改变，旧阈值与新数据可能不再可比。例如，同一个“拥堵”标签若一批记录按现场观察填写，另一批改由自动计数器产生，标签误差和输入尺度都可能变化。此时不能只把新旧样本合并再报一个分数；应先记录定义变化发生的时间和范围，判断哪些数据仍可比较，再决定沿用旧规则、分段分析或重新采集。规则版本应与字段说明、适用对象和验证记录一起保存。

模式识别可以由人手工完成，也可以借助程序或 AI 搜索规律。AI 可能从样本中列出一条得分很高的规则，却不一定知道字段是在预测前还是预测后产生，也不

一定知道某个记录是否来自同一对象的重复测量。让 AI 提出候选时，应要求它标出使用字段、阈值、正负样本、被覆盖的反例和未知边界；再由人确认字段含义和生成时间。候选规则应能够被另一位读者按原始记录重算，而不是只交一段无法追溯的自然语言解释。

2.3.4 开发和测试必须隔开

用于构造规则的样本和用于估计规则表现的样本，承担不同作用。开发集用于比较候选、选择阈值和修订规则；测试集用于在候选与判断标准冻结后估计未见数据上的表现。如果看过测试结果后再改规则，测试集已经参与开发，不再是独立的检验依据。需要再次估计时，必须收集新的未使用数据，或重新设计适合当前问题的验证方案。

即使没有查看测试集，开发者若尝试几十种字段组合和阈值，再只汇报开发集上最好的一个，分数也可能高估规则的稳定性，因为候选选择本身已经适应了有限样本。应记录候选范围、选择依据和最终冻结时刻；在资料稀少时尤其要避免把每个偶然差异都变成一条特例。独立测试用于检查选中的完整流程，而不是为寻找下一个“最好规则”提供无限次提示。

“隔开”不是只在表格中画一条分界线。若数据含有同一对象的多条记录，随机按行拆分可能让同一对象的一部分进开发集、另一部分进测试集。模型便可能认出对象特征，而不是学会对新对象有效的规则。若目标是预测未来，随机混排时间记录也可能让早期训练过程间接利用未来分布的信息。应按实际部署问题决定分组

或时间切分：要泛化到新个体，就让同一对象的记录留在同一侧；要面对未来，就用过去开发、后续时间测试。[3]

切分方式还决定了“新数据”究竟有多新。若未来部署面对同一批对象的下次观察，按对象隔离可能过于严格；若部署面对从未出现过的对象，只按时间切分又可能让同一对象泄漏到两侧。应把目标说具体：预测下一时段、预测新个体，还是同时预测新个体和新时期。然后用与目标相符的分割方式验证。把数据拆得整齐并不自动构成好评估，检验情境必须对应实际要作出的判断。

处理数据的步骤也可能泄漏信息。假设在全部记录上先计算平均值和标准差，再把标准化后的记录分成开发与测试，测试样本已经参与了转换参数的估计。缺失值填补、特征筛选、降维、阈值选择等操作都要放在每次开发数据内部拟合，再将同一转换应用于相应验证数据；官方实践指南特别提醒，先拆分再拟合变换能够避免测试信息进入训练流程。[4]

泄漏还可能来自结果定义。若系统要预测未来拥堵，却把拥堵发生后才记录的排队长度放进输入，表面分数再高也不能支持上线预测。检查每个字段时，可问三个问题：它在预测时刻之前是否已经生成？它的记录来源是否独立于目标标签？部署时能否以相同含义及时取得？如果不能，应该删除、重定义任务或明确将其改为事后诊断，而不是继续报告同一预测结论。

2.3.5 分数要连同分母和错误代价一起阅读

对二分类结果，可以把预测和实际标签交叉，区分四种计数：真正例 TP 是实际为正且预测为正；假负例 FN 是实际为正却预测为负；假正例 FP 是实际为负却预

测为正；真负例 TN 是实际为负且预测为负。准确率为(TP+TN)/全部记录，精确率为 TP/(TP+FP)，召回率为 TP/(TP+FN)。每个比例都要报告分子和分母；若分母为 0，该指标未定义，不能随意记作 0 或 100%。

只报告准确率可能隐藏重要失败。若正类很少，一个总是预测负类的规则也可能有很高准确率，但对任何正类都没有召回。反过来，扩大正类覆盖常会增加误报。选择规则前应先确定错误的实际代价：错过一次拥堵预警与多发一次提醒，影响可能不同；偏好哪一类错误要由场景责任人和受影响者共同讨论，不能从一个总分自动推出。

小测试集上的一个样本就可能显著改变比例。四条记录中错一条与四百条中错一条，不具有相同的信息量。小样本可用于发现明显反例、检查计算与提出后续采集问题，却不足以证明总体性能、统计显著性或跨场景优势。描述结论时应限定为“在这批记录上观察到……”，再说明需要怎样的新数据验证。

阈值选择还包含行动后果。把预测分数转换成“提醒”或“暂不提醒”，不是纯数学步骤：阈值降低通常会覆盖更多可能的正类，同时可能增加误报；阈值升高可能减少提醒，也可能漏掉需要处理的情况。若不同错误会引发不同代价，可报告多项计数或比较不同阈值的代价情景，而不是把某一个指标称为唯一客观标准。一个阈值在纸面上合适，也要检查现场是否能采取后续行动；无法响应的告警不应被视为完整解决方案。

2.3.6 练习 E03：从有限记录构造拥堵规则

以下记录是原创教学模拟数据，不代表校园实测。目标是在预测时点判断某入口未来 10 分钟是否拥堵。x 是预测时点前 5 分钟已观测的人数，event 表示当时已知是否安排活动，y 是随后 10 分钟实测的拥堵标记（1 为拥堵，0 为未拥堵）。所有记录来自不同场次，按时间从 d1 到 d12 排列。d1—d8 为开发集，d9—d12 为冻结测试集。不得把 y 作为预测输入。独立学习时，可先遮住测试集四行；写下开发集比较结论后再查看。

表2-5 E03 模拟观察记录

记录	x（人）	event	y
d1	80	0	1
d2	70	0	1
d3	55	0	1
d4	45	0	0
d5	30	0	0
d6	60	1	1
d7	40	1	1
d8	20	1	0
d9	65	0	1
d10	48	0	1
d11	38	1	0
d12	25	0	0

来源：本书原创教学材料与作者推演；模拟数值的条件见对应题设。

比较两个预先给定的候选规则：RA 为 x >= 50；RB 为 x >= 50 或 event = 1 且 x >= 35。先只用 d1—d8，分别计算 TP、FN、FP、TN、准确率、精确率和召回率，并记录你会选择哪一条规则及其理由。然后按原规则对 d9—d12 作预测，重新计算计数和指标，不依据测试结果回头改阈值。解释这四条记录支持和不能支持的结论。

再指出一个足以挑战所选规则的具体记录或边界变化，说明它改变了哪条条件判断。提出一种可在预测时点追加采集的信息，写清字段含义、测量时刻以及它可能补充的区分；不能把预测之后才产生的结果当作输入，也不能预先宣称新字段一定提高表现。最后构造一个若按随机行拆分就会造成泄漏的“同一对象多次记录”例子，以及一个未来数据泄漏的时间顺序反例。

成果为一张开发与测试混淆计数表、一段有限结论、一条反例、一项新增信息方案和两种切分反例。评价重点是没有混用数据、计数完整、指标分母正确、知道小样本限制，并能把规则与可用信息的时间点对应起来。规则在某组数据上得分更高，不等于它已经被证明普遍更可靠。

参考推演见章末“E03参考解与讨论”；请把开发阶段选择与测试后的结论分别写下。

【本节反思】你看到的一条“规律”是重复取值、事件结构还是统计关联？如果新例外出现，你会先检查数据、条件、表示还是规则范围？说明判断依据。

2.4 算法设计：搜索、排序与决策过程

一条路线上有许多岔口时，人很容易把“眼前看起来最省事”当成“最后最省时”。对计算机而言，问题更具体：什么信息代表当前处境？哪些后续处境值得继续检查？何时可以确定某条路线已经最好？如果我们把答案写成程序，这些判断必须变成能执行、能检验的步骤。算法设计的难点不在于把流程写得足够详细，而在于让流程求解的目标与原问题一致，并说明它依赖哪些条件。

本节用一张小型有向路网考察不同搜索策略。设配送员从 P 点出发，终点为 D；各条道路时间是固定的分钟数，图中没有列出的道路不存在。边权都非负。路线信息如下：

表2-6 六节点示例的有向道路

起点	可直接到达的地点与时间（分钟）
P	A：2，B：3
A	C：6，D：11
B	C：2，E：5
C	D：2
E	D：3
D	—

来源：本书原创教学材料与作者推演；模拟数值的条件见对应题设。

这是原创教学示例，不代表真实校园路网。它故意让“少走几段路”“每一步选最小边”“最终用时最短”给出不同答案。若一个方法只在看上去顺手的例子里成立，学生还不能据此判断它适合什么任务；需要追踪状态，并指出证明结论所依赖的条件。

2.4.1 把问题写成状态、转移和目标

算法首先要有一个明确的“当前状态”。在上面的静态路网中，状态可以只是所在地点；从 P 到 A 的一次移动就是一条状态转移，其代价为 2 分钟。一条候选路线是若干转移的序列，路线总时间是边权之和。任务目标也要说清：到达 D 即可，还是必须经过某个地点？要最少经过路口，还是最少耗时？如果未先回答这些问题，“最优”没有可检验的含义。

把问题简化成“地点”是否足够，取决于地点之外的信息会不会改变后续选择。假设配送员必须先领取门禁卡，才能进入机房。此时同在 A 点但“尚未领卡”和“已经领卡”不是同一状态：两种情况允许的后续转移不同。可以将状态写为“（地点，是否持卡）”。若某项信息不改变可选动作、转移代价或目标判断，它或许不必加入状态；若它改变其中任何一项，合并这两种情况就可能删除真实可行路径。状态设计由问题规则决定，而不是由地图上节点的名称决定。

一个搜索问题通常可以拆成五个彼此关联的部分：初始状态、可用动作、状态转移、代价或偏好、目标条件。还需说明哪些状态禁止进入、目标是否唯一，以及没有路线时应如何报告。动作和转移把现实规则变成可执行关系；代价说明候选方案如何比较；目标条件确定何时完成。缺一项，程序仍可能输出某个结果，却无法说明它是不是用户要求的结果。

状态模型还决定算法究竟在多大的空间里搜索。假如路线受到“剩余电量”的限制，单记录地点会把电量充足和即将耗尽两种处境合并；加入电量后，同一个地点可能对应许多状态。状态越细，越能区分后续选择，但节点数也可能大幅增长。减少搜索的剪枝规则因此需要证明：被剪掉的状态确实不会产生优于保留状态的

解。常见做法是比较到达同一地点的候选：只有当一个候选在所有相关资源上都不差、代价也不高于另一个，才能考虑将后者排除。只看当前已花时间少就丢弃电量更多的路径，可能删掉唯一能到终点的方案。

现实代价还可能随时间变化。若边 P→A 在不同时段的耗时不同，给这条边固定一个数字就无法同时代表早高峰和深夜。可以把到达时间并入状态，并按当前到达时刻计算下一段耗时；若又把时间粗略分成“早/中/晚”，这会降低状态数量，却可能把同一时段内的重要差异抹掉。必须说明这种离散近似会不会改变可行性或路线比较。更改状态、代价或转移规则后，原来适用于静态路网的算法证明不一定仍成立。

搜索过程会维护一组尚待展开的候选状态，称为搜索前沿。处理一个状态时，算法检查它能到达的新状态，再按特定规则把候选放进队列或优先队列。已访问集合用于避免在有限图中反复展开同一状态；不过，若带有资源或任务进度等状态信息，只能合并完全相同的状态，不能仅因地点相同就标记为“已经访问”。可达状态有限、每个状态只按规则有限次入队并展开时，搜索才有终止依据；目标不可达也应最终报告无解，而不是无限等待。

2.4.2 广度优先搜索回答的是“最少几步”

广度优先搜索（BFS）把起点放入先进先出队列。每次取出队首状态，按规则枚举相邻状态；第一次发现的新状态被记下前驱并放到队尾。因为队列按层处理，距起点一步的状态先于两步状态，随后才是三步状态。若目标是最少转移次数，且每次转移代价相同，BFS 第一次发现目标时得到的就是步数最少的路线；在按边的

数量计费且所有边费用同为常数时，这也对应总费用最少。其队列顺序构成正确性的关键，而不只是实现细节。[5]

“第一次找到”能证明什么，取决于队列表达了什么顺序。在本节的路网中，先按字母顺序枚举邻点，BFS 会先从 P 发现 A，再从 A 发现 C 和 D。因此它得到 P—A—D，两段道路、总时间 13 分钟。另一条 P—B—C—D 需要三段。对“最少经过几段道路”而言，两段确实比三段少；对“最短用时”而言，13 分钟并不比 7 分钟优。BFS 没有算错，它忠实优化了步数。错误发生在把它的目标偷换成最短时间。

如果边的代价不相等，把队列规则改成按当前已知总时间排序，才有机会优化加总代价；此时需要额外的正确性条件和证明。若某一步包含等待、换乘、拥堵状态或是否携带设备等信息，单纯节点图也可能不足。比如一条边通常走 2 分钟，但在“闭馆后”不能通过；这时状态还要包含时间阶段，或者明确将该边视为不可用。先把问题表示正确，再谈选择搜索方法，能避免把某个熟悉算法套在不满足前提的表示上。

BFS 的分层结构也给出一个简单的不变量：队列中更早进入的状态不会比后进入的状态离起点更远。起点距离为 0；处理距离 k 的状态时，只会把首次发现的相邻状态放在距离 k+1 层。已发现标记阻止环路让同一状态不断入队。由此，首次发现某目标时，不会还有一条经过更少边数、但尚未处理的路线。若每条边费用相同，边数与总费用成固定比例；若边费用不同，层数仍正确，但它不再代表时间成本。比如前述路网中，P—A—D 虽然只有两段，耗时却是 13 分钟，而三段路线 P—B—C—D 只需 7 分钟。BFS 的结论边界由不变量精确限定。

2.4.3 用 Dijkstra 算法处理静态非负代价

当边的时间不同，但每条边代价固定且非负时，可以用 Dijkstra 算法求单源最短路径。原论文讨论正长度的边；现代教材中常见的非负权版本也允许零权边。[6][5] 它为起点设置暂定距离 0，其余状态先设为无穷大；每轮选取暂定距离最小、尚未确定的状态，将它的距离确定下来，再用它更新相邻状态。这里“暂定”表示已经发现一条真实路线，但还没有证明不存在更短路线；“确定”表示按算法条件可以证明当前距离已经是最短距离。两者不能混为一谈。

对本节六节点示例，开始只有 P 的距离为 0。确定 P 后，A 和 B 分别得到 2、3。确定 A 后，C 得到 8，D 得到 13。此时再把 B 确定下来，经 B 到 C 的距离为 3+2=5，比已有 8 更小；到 E 的距离为 3+5=8。随后确定 C，经 C 到 D 的距离为 5+2=7，替代已有 13。达到 D 时，算法可以停止并给出路线 P—B—C—D、总时间 7。表 2-7记录了每轮处理之后的暂定距离；没有列出的状态仍为∞。

表2-7 六节点示例的 Dijkstra 推演

确定的状态	本轮可改进的距离	本轮之后尚未确定的关键距离
P（0）	A=2，B=3	A=2，B=3
A（2）	C=8，D=13	B=3，C=8，D=13
B（3）	C 由 8 改为 5，E=8	C=5，D=13，E=8
C（5）	D 由 13 改为 7	D=7，E=8
D（7）	到达目标，终止	E=8（未处理）

来源：本书原创教学材料与作者推演；模拟数值的条件见对应题设。

注：∞表示尚未发现从起点到该状态的路线。

图 2-5把“已经确定”和“仍在前沿等待比较”的状态放在同一张图中。读图时应跟踪一个事实：边的终点可能先得到一条较长的暂定路线，之后被另一条更短路线更新。若把“第一次发现”误当作“已经最优”，就会停在 D=13；优先队列的用途正是每次取出当前最小暂定距离，让这些后续改进有机会发生。

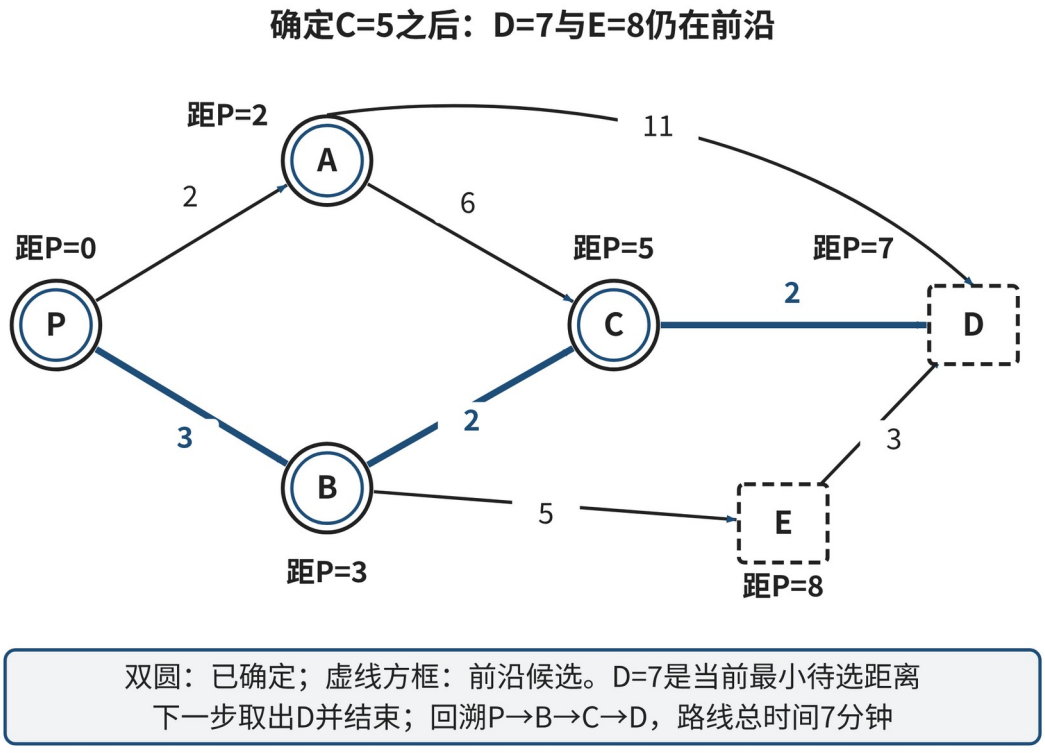


图2-5 路径搜索中的前沿状态与暂定距离

说明：边旁数字表示分钟，节点旁数字表示从 P 到该点的距离；节点样式和线型区分已确定状态与仍在搜索前沿的状态。该图对应正文独立讲解示例，不对应练习 E04。

来源：作者自绘。

这个停止规则需要证明。若当前取出的最小暂定距离状态 u 仍有一条更短的真实路线，那么沿那条路线从起点向 u 前进，必有一个首个尚未确定的状态 x，它的前一个状态已确定。此前驱被处理时，算法已检查通往 x 的边，因此 x 的暂定距离

不大于这条更短路线到 x 为止的前缀代价。由于剩余边权非负，前缀代价不大于整条路线，也就小于 u 的暂定距离。这与 u 是前沿中暂定距离最小者矛盾。因此，u 出队并被确定时，它的距离已经最优。证明使用了“所有边权非负”；如果有负边，路径走得更远后总费用仍可能下降，前缀顺序便不再支持这个推理。[6]

使用二叉堆实现优先队列时，稀疏邻接表上的 Dijkstra 通常需要约 $O((|V|+|E|)\log|V|)$时间和 $O(|V|+|E|)$空间；使用数组扫描而不用堆，选择最小暂定距离可能需要 $O(|V|^2)$时间，稠密图上实现简单且未必更差。BFS 使用邻接表时为 $O(|V|+|E|)$时间，队列和访问标记为 $O(|V|)$额外空间。算法选择也包括表示与数据结构的取舍：堆会增加维护成本，稠密矩阵则可能占用 $O(|V|^2)$空间。实际比较还取决于图的规模、稀疏程度和边是否需要临时过滤。[5]

优先队列实现还有一个值得检查的细节：某状态的暂定距离可能被改小，旧的较大键值仍留在堆中。常见实现可以在堆中插入新值，取出时若它已不等于当前记录，就跳过这个过期条目；也可以使用支持降低键值的数据结构。无论采用哪种方式，都要保证“被确定”的确是当前最小有效暂定值。读代码时若看到同一状态多次入队，不应立刻判断为错误；应追问旧条目如何识别、是否会重复展开，以及复杂度分析是否包含这些操作。

零权边仍满足非负条件，Dijkstra 的距离正确性不因此失效，但多个等距前驱会让路径本身不唯一。实现者仍须决定如何处理等距候选、怎样稳定输出前驱，以及何时停止。负边则不满足标准 Dijkstra 的保证；有负权环时，某些路线可反复绕行而使总代价不断下降，最短有限路线可能不存在。若模型允许负边，需要改用满足相应条件的算法，或重新检查代价定义；不能只把 Dijkstra 的符号改一改。

下面的伪代码把关键规则连在一起。它以静态非负边权、已按通行条件过滤的图为输入；堆以（距离，节点字母）比较。dist 保存目前发现的最小代价，prev 用于回溯路线，done 保存已经确定最优距离的节点。

```
所有 dist 置为无穷；dist[start] = 0
prev = 空映射；done = 空集合；heap = [(0, start)]
while heap 非空:
    (d, u) = 弹出最小项(heap)
    if d != dist[u] or u in done: continue
    将 u 加入 done
    if u == target: 返回 (d, 由 prev 回溯的路线)
    for 每条合法出边 (u, v, w):
        if v not in done and d + w < dist[v]:
            dist[v] = d + w; prev[v] = u
            将 (dist[v], v) 插入 heap
返回“目标不可达”
```

严格小于才更新使等距候选保留先发现的前驱。只从 start 入堆可以避免把不可达节点当作待展开对象；起点就是目标时，回溯得到只含起点的路线。把目标判断放在发现邻点的位置，会把暂定结果误当成最终结果。这里的代码采用旧条目跳过法，堆可能保存 O(|E|)个条目；对无平行边的简单图，log|E|为 O(log|V|)，与前述时间量级一致。若允许任意多条平行边，就应按实际堆大小写成 O((|V|+|E|)log(|V|+|E|))，不能省略输入表示的条件。

目标的最短距离确定后，最优路线仍可能不唯一。算法若只记录距离，仍能回答“最短时间是多少”，却不能回答“哪条路线更方便”；若要偏好少转弯、避开台阶或减少换乘，需要把这些偏好加入比较规则或状态信息。按节点字母打破平局可以让程序每次输出一致，但这只是可复现的技术约定，不意味着字母顺序在现实中更公平或更适合用户。算法的确定性与决策理由应分开说明。

2.4.4 局部贪心为何会走错方向

回到示例路网。如果配送员每到一处就选当前可走且最短的一条边，并且不回头，那么从 P 会先选 A（2 分钟）；在 A 选 C（6 分钟，而到 D 需 11 分钟）；再从 C 到 D（2 分钟），总计 10 分钟。算法每一步都作出了局部最小选择，但没比较尚未走过的另一条支路。Dijkstra 则把所有已发现路线放在同一前沿中比较：P—B 的 3 分钟暂定距离早于 A—C 的 8 分钟被处理，进而发现总计 7 分钟的路线。

这说明“贪心”不是一个统一算法，而是按某种局部指标选择下一步的策略。要证明局部选择能导向全局最优，必须说明它具有适合该问题的交换性质或其他结构，不能从“每一步都不浪费”直接推出“整体最佳”。贪心在某些问题上正确，在另一些问题上会失败；失败也不说明局部选择毫无价值，它可能适用于近似、排序或特定约束问题，只是需要单独说明目标和保证。

路径任务常见的三个问题可以互相区分：BFS 在统一边代价下找最少步数；Dijkstra 在静态非负边权下找最小累计代价；局部最小边策略每次只看当前位置的直接边。第三种策略没有把搜索前沿和累计距离放在一起比较，因此容易把“眼前便宜”误当成“从起点到终点总价低”。算法名称不会自动给出适用条件，能否匹配目标要看状态、转移、代价和停止条件。

2.4.5 排序也是决策过程的一部分

路径搜索中，候选处理顺序影响什么状态先被检查；很多非图任务也会通过排序决定先处理谁。排序不是无害的展示操作：若后续程序遇到同优先级对象时采

用“第一个”或“最后一个”，排序顺序会改变决策结果。因而需要说明关键字、方向、平局规则、稳定性和缺失值如何处理。

考虑整理一组图书目录，类别顺序由任务规定为“传记”在前、“科普”在后，每类内部按出版年份从早到晚排列。样例数据为：甲（传记，2018 年）、乙（科普，2022 年）、丙（传记，2020 年）、丁（科普，2019 年）。复合键（类别顺序，出版年份）得到甲、丙、丁、乙。所谓字典序，是先比较第一项，只有第一项相等才继续比较第二项；它把口头上的“先……再……”变成确定规则。

同一结果也可由两次稳定排序得到：先按出版年份升序，得甲、丁、丙、乙；再按类别顺序稳定排序。稳定排序会保留同一类别记录在上一步中的相对次序，于是传记类为甲、丙，科普类为丁、乙，最终为甲、丙、丁、乙。复合键（类别顺序，出版年份升序）也得到相同次序。[7]

排序前就要明确偏好：上例按指定类别顺序分组，再在组内按出版年份升序；若资料馆希望新书在前，第二关键字就要改为年份降序。还需约定类别或年份都相同时，是否保留输入记录顺序。稳定性只保证关键字相等时不打乱原有相对顺序，不会自动替学生决定平局应该意味着什么。多轮排序时，先排次要关键字，再用稳定排序排主要关键字；复合键直接编码所有比较规则，更便于读者审查。

2.4.6 让 AI 生成候选，再检验算法主张

生成式 AI 可以根据自然语言需求草拟伪代码、解释复杂度或补充测试数据，但输出首先只是候选方案。算法是否正确，不能由说明文字流畅、样例运行成功或“看起来像标准写法”来判定。需要把检查拆成可答的问题：状态是否保存了影

响后续决策的信息？优化目标是否与题目一致？入队、更新和已访问规则是否匹配算法？边界情况下能否终止？声明的复杂度对应什么数据结构？

一个有限的协作过程是：先让 AI 把需求改写成输入、输出、状态、约束和停止条件；由人确认其中没有丢失必要条件；再要求它给出伪代码和每个关键变量的含义；最后用手算小例、反例和自动化测试分别检查常规情况与边界。对本节的路线搜索，至少应检查起点即为终点、目标不可达、图含环、有相同暂定距离、边权为零，以及算法不允许负边的情况。对于排序，还需检查重复关键字、并列值和不同升降序。边界测试不是“多跑几个样例”而已：每个用例都要针对一项规则或假设。

AI 也可能把目标写错后仍生成内部一致的程序，例如把最少跳数误称作最短时间，或看到目标第一次出现便提前停止。阅读程序时，应从输入追踪一次状态更新，确认每条边只在规则允许时使用，核对输出路径的相邻节点确有边、累计代价确实等于各边之和。若程序给出路线但无法说明它优化了什么，先回到问题契约，而不是再要求模型把代码改得更漂亮。人保留目标设定、条件核对和结果解释的责任；AI 适合加快候选生成和审查，不替代正确性证明或独立检验。

2.4.7 E04：比较三种路线策略

以下邻接表描述一幅原创教学图。所有道路有向、时间单位为分钟且静态；边权非负，其他边不存在。图的邻接点按字母顺序枚举。

表2-8 E04 有向道路

起点	可到达地点与时间
S	A：1，B：4
A	B：2，C：1，T：9
B	T：2
C	T：9
T	—

来源：本书原创教学材料与作者推演；模拟数值的条件见对应题设。

任务：分别模拟 BFS（邻接点按字母顺序枚举，目标 T 出队时停止）、当前位置选最小可用边且不重复经过节点的局部贪心策略（边权相同按字母顺序），以及按最小暂定距离确定节点的 Dijkstra 算法（距离相同时按字母顺序）。记录每种方法的访问或确定顺序、返回路线和总时间；Dijkstra 逐轮记录暂定距离及前驱。然后把边 B→T 的时间改为−3，判断这是否违反标准 Dijkstra 的条件，并说明原图改动后算法结果本身是否必然错误；再给出顶点和边尽量少的有向图，证明标准 Dijkstra 会确定错误距离。不得把负时间解释成现实路线，只用它检验算法假设。

另有四项待处理记录，priority 越小越先处理：r1（priority=2，submitted=09:03）、r2（1，09:02）、r3（2，09:01）、r4（1，09:01）。先按 submitted 升序稳定排序，再按 priority 升序稳定排序；写出每轮次序并与一次复合键（priority 升序，submitted 升序）排序结果比较。说明若 priority 和 submitted 都相同，稳定性保留了什么信息，以及若要求新的平局规则应修改哪里。

提交物：三种搜索方法的状态记录、每条返回路线的代价核算、负边变体说明、稳定排序的两轮中间结果。评价看：目标和假设是否写清；距离更新及前驱是否一致；步数与代价是否区分；反例是否确实推翻“负边也能用标准 Dijkstra”；排序是否正确执行了复合规则。只报路线名称或程序输出不构成完整论证。可以手算，也可提交短程序；程序须说明数据结构，并对未设定或不可达的情况给出可读处理。

本题的参考推演见章末“E04参考解与讨论”；请先保存自己的逐轮记录，再比较差异。

2.4.8 反思：结果正确的依据是什么

回顾你选用的搜索和排序规则：题目中的“最好”具体指什么？把状态从“所在节点”扩展为“节点加一个条件”后，访问标记和路线是否需要变化？如果图上每条边都相同代价，你的选择是否更简单？如果某个比较关键字缺失或平局策略改变，结果会如何变化？指出一条能够区分两种算法的最小反例，并说明它检验的是目标、状态、边权假设还是停止条件。回答这些问题，能把“程序运行了”进一步变成“知道它何时能回答这个问题，以及答案凭什么可信”。

2.5 评估、优化与自动化：建立反馈闭环

两种方法都输出了路线，怎样判断哪一种更好？一种方案给出的成功路线更快，却更经常拒绝行动受限访客；另一种查询速度更快，却需要较长的预处理，而且地图一变就要重做。评估应当把这些差异放到同一个任务中解释。本节从“平均值改善”的疑问进入，连接比较指标、优化成本和自动执行的前提。

2.5.1 平均值下降为什么仍可能不宜采用

讲解示例：不同分母的平均。在一组原创模拟请求中，基线为五个请求返回路线，耗时分别为 6、7、8、9、10 分钟；候选只为前三个请求返回路线，耗时为 5、6、7 分钟，另外两个没有合格结果。成功请求的平均时间从 8 降到 6 分钟。这个计算没有错，但它不能单独支持“候选对这五个请求更好”：两种平均值比较的对象不同，候选没有成功的请求被移出了分母。

如果服务要求所有请求都有合格结果，候选已经违反要求。如果允许部分请求转交人工，需要同时记录覆盖率、转交原因和额外等待，而不是把失败行删除。报告“平均 6 分钟、成功 3/5”使读者能够判断代价；报告“缩短 25%”却不说明覆盖变化，会把一个有限指标包装成整体改善。评价错误有时发生在计算前的样本选择，而不是计算公式本身。

可以先将结果按请求 ID 配对，比较两种方法共同成功的那部分，再单独报告只在一种方法上成功的请求。配对比较回答“对共同成功的请求，时间怎样变化”，覆盖比较回答“有多少请求得到结果”。两者不能互相替代。若希望计算一个综合量，必须明确怎样处理失败，例如给每个未完成请求赋一个约定的等待代价，并解释这个代价的依据。这样的量是任务方定义的评价规则，不是失败在现实中必然产生的准确损失。

某个群体的结果也可能被总体平均遮住。假设多数普通步行请求得到小幅改善，少数无台阶请求的失败却增加，总体分数可能仍然变好。因而需要按实际需求分组查看，而非先假定一个总分足以代表每个人。分组也不应无限细到无法判断；

应优先检查具有不同硬条件、不同失败成本和不同数据覆盖的群体。对本章导览而言，是否要求无台阶通行就是一个具有语义依据的分组。

2.5.2 先确定比较单位，再选择指标

比较单位可能是一个请求、一条道路、一场活动或一次系统调用，不同单位对应不同问题。若一个访客连续发起十次查询，把十次当成十个独立用户会夸大样本多样性。若测试目标是每次调用的计算耗时，则十次调用又可以是合法记录，但要说明它们共享用户与地图。评价不是禁止重复测量，而是要知道重复来自哪里。

本章采用三层检查。第一层检验硬条件：方向、通行要求、必须经过的地点、截止时刻与版本是否满足。第二层比较任务代价：行走时间、绕行、拒绝与等待。第三层比较实现成本：计算时间、预处理、内存与维护。先确定第一层，才能防止用更快的错误路线击败稍慢的正确路线。第二、三层也不能混为一谈；少算几毫秒不会直接让访客少走几分钟。

指标须附单位、统计范围和失败处理。平均查询时间容易计算，但少数很慢的请求可能影响交互；最大值能显示本批极端情况，却容易受单次异常影响。中位数描述本批中间位置，不能显示全部尾部。可以报告这些量的组合，并查看原始记录。小样本的分位数取法不同会得到不同值，教材中的小表不适合据此制造精确的“99%请求保证”。

指标之间的冲突需要决策原则。有些条件是门槛，例如确认无台阶通行；有些是门槛通过后的优选标准，例如平均时间。把门槛乘一个权重与其他指标相加，可能让足够大的速度收益抵消通行失败，改变原任务。若任务确实允许这种补偿，须

先说明并取得相应领域决策依据，不能由程序暗中改变。本节练习的硬门槛直接给定，学生无需猜测权重。

当多个方案在不同指标上各有优势时，可先判断是否存在支配关系：一个方案在所有约定指标上均不更差，并在至少一个上更好，则另一个没有这些指标上的优势。没有被支配不表示应当采用，只说明还需取舍；被支配也依赖所选指标和适用条件。遗漏维护成本或某类请求，可能改变支配关系。因此，多指标表是帮助看清选择的工具，不是消除价值判断的机器。

2.5.3 同批、同条件和保留的测试

比较两种路径策略时，应当固定输入图、边权、通行条件、请求集合和失败定义。若基线在早高峰地图上运行，候选在平峰地图上运行，差异同时包含算法与环境变化。对静态模拟图，可以保存同一快照并重复运行；对真实道路，环境会变化，需要另行设计采样与比较。这两类证据的强度不同，不能把离线模拟成绩称为现场效果。

NIST 的 AI 风险管理框架要求记录测试集、指标和测试工具，说明与使用情境相近的评价条件，并记录适用范围的限制[8]。本节将这项要求落实为一个很具体的习惯：在比较表旁保留输入版本、请求 ID、约束版本、策略版本和运行方式。不是为了堆积文档，而是使一个异常结果能够被定位和复算。没有这些信息，后来换了地图还看到旧分数，便无法知道分数属于谁。

同一批五个请求：时间与覆盖分开看

讲解示例数据；“失败”表示没有合格路线，不计入成功平均

请求	基线B	候选C	配对状态	分组
R1	6分	**5分**	共同成功	未提供
R2	7分	**6分**	共同成功	未提供
R3	8分	**7分**	共同成功	未提供
R4	9分	失败	仅B成功	未提供
R5	10分	失败	仅B成功	未提供

成功均值：B=(6+7+8+9+10)/5=8分钟；C=(5+6+7)/3=6分钟

覆盖：B 5/5
C 3/5

共同成功R1–R3
C比B各快1分钟

本例未提供通行条件分组标签；真实分组需另有数据，不能由图推造

图2-6 配对比较、覆盖与分组评价

来源：本书原创教学图；图中数量来自 2.5.1 五个请求的讲解示例，与 E05 数据分开，不表示实地服务性能。

图 2-6应从共同的五个请求开始读，而不是只看平均值。观察每种方法在哪些行成功，再检查平均时间用了哪些行。共同成功的前三行可以配对比较，另外两行的失败必须另报；具有不同通行条件的请求还应分组检查。图表把不同分母显式分开，使“速度更快”与“服务更多请求”能够分别接受判断。

开发集用于发现问题和选择方案；测试集用于在选择完成后检查新材料上的表现。反复根据测试失败修正候选后，那批数据就逐渐变成开发材料。可以保留它检验已知问题是否复现，却不能继续把同一成绩当成完全未见数据上的证据。 2.3 讨论的规则归纳有同样边界；此处扩大到完整路线策略与评价过程。

没有新数据时怎么办？可以诚实报告“改动消除了已知反例，在这批回归材料上没有发现新失败”，同时说明尚缺独立泛化证据。不能把缺少测试当作无法改进，也不能通过改名把旧集重新变成独立集。新测试应针对未覆盖条件，例如不同图结构、相同代价的多条路线、入口临时关闭与没有可行路径，而非只改变已有记录的编号。

比较方法也应控制自己的代价。若在一个六节点确定图上可以穷举全部简单路径并得到精确最优值，就可以直接检查候选是否最优，不必先做一个含糊的“用户更喜欢哪种算法”的测试。若某成熟算法的适用条件已经成立，应优先采用已有可靠方法，再用当前数据验证实现。只有对当前场景仍有实际未知，例如动态预测误差与交互行为，才需要针对未知设计额外比较。验证要回答问题，不能把实验形式本身当成果。

2.5.4 优化要计算准备成本和失效条件

讲解示例：预处理是否值得。某策略每次查询需要 30 毫秒；另一策略先花 90 毫秒建立索引，以后每次需要 12 毫秒。对 q 次查询，前者总耗时为 30q，后者为 90＋12q。只有 q 大于 5，后者在这项总耗时上严格更快；q 等于 5 时相同。若地图每次查询前都变化，索引不能复用，那么每次都要重新支付 90 毫秒，原先的优势消失。

这个推导把“更快”拆成了三个条件：测量哪些操作、重复多少次、准备结果可以复用多久。现实优化经常依赖这种摊销。建立缓存、排序后查找、预计算常用路线，都可能把一次成本换成多次收益；反过来，输入变化、内存约束或缓存验证

也可能抵消收益。不能只展示优化后最便宜的那一步，把必须发生的准备工作藏起来。

缓存的键应包含会改变结果的全部输入。对本章路线，起终点相同但通行条件不同，不能复用同一结果；地图版本改变，旧路线也不能自动继续有效。缓存键中加入 profile 和 map_version，只能减少遗漏这些已识别输入的风险，不能证明真实通行信息完整。如果底层事实没有更新，缓存甚至会更高效地重复一个过时错误。

可以从两个维度识别瓶颈：某一步单次占用时间大，或某一步被反复执行很多次。一个解析步骤只要 2 毫秒，却重复一万次，可能比一次 100 毫秒的搜索更值得优化。先观察调用次数与总耗时，再决定是否改算法或减少重复。只根据函数看起来复杂就优化，可能投入很多努力却几乎不改变完整任务的时间。

优化还可能改变结果。剪枝减少搜索，但只有在不删除必要候选的条件下才能保留完整性；近似方法可能更快，却有不同的误差或最优性承诺。比较时应说明输出保证是否变化。例如“在当前非负图上得到精确最短路径”与“快速得到一条可行路线”是不同成果，不能只用运行时间一列比较。若允许近似，应记录误差怎样定义、怎样估计、哪些请求需要回退到精确方法。

2.5.5 从计算过程到自动执行

自动化意味着某些步骤不再逐次由人操作，但它仍然需要输入条件、状态和结束规则。计算一条候选路线与向访客发布“此路线现在可用”是两个不同动作。后

者包含对证据时效和输出责任的承诺。为了提高自动化程度而省略这些条件，会把模型内可行偷换成现实已确认。

可以先把过程分为准备、检查、提交和确认。准备阶段生成候选；检查阶段验证当前输入和全部硬条件；提交阶段产生外部效果；确认阶段判断效果是否发生。若检查失败，应返回具体原因；若提交后没有收到确认，应保存“结果未知”。未知不是肯定失败，也不是肯定成功。这个区别对任何带外部作用的自动流程都重要。

讲解示例：通知是否应重发。系统向某访客发送路线更新，网络在发出请求后断开。服务器可能已经发送，客户端只是没拿到确认；也可能请求根本没有到达。直接重发会在第一种情形中产生重复通知。若系统可以按同一个操作 ID 查询已发送记录，便可先取得证据；若没有这样的能力，应暂停该项并交给明确的恢复处理，而不能用“多试几次总会成功”消除不确定性。

幂等性描述重复相同操作对约定目标状态产生与一次操作相同的效果。RFC 9110 对 HTTP 方法给出这种语义，并指出非幂等请求的自动重试需要额外依据，例如知道原请求未被应用或其实际语义可幂等[9]。这里借用的是重试判断的原则，不将某个协议方法名当成应用业务绝对安全的证明。即使最终状态相同，重复请求仍可能产生日志或其他附加作用。

给通知附操作 ID 还不够。接收方必须真正依据该 ID 识别同一次业务动作，并明确多久保留记录、重复请求返回什么、并发请求如何处理。若每次重试生成一个新 ID，系统就把同一动作伪装成了两次新动作。教材不要求实现完整分布式系统，但你应能指出自动流程需要怎样的契约，才能证明某次重试有根据。

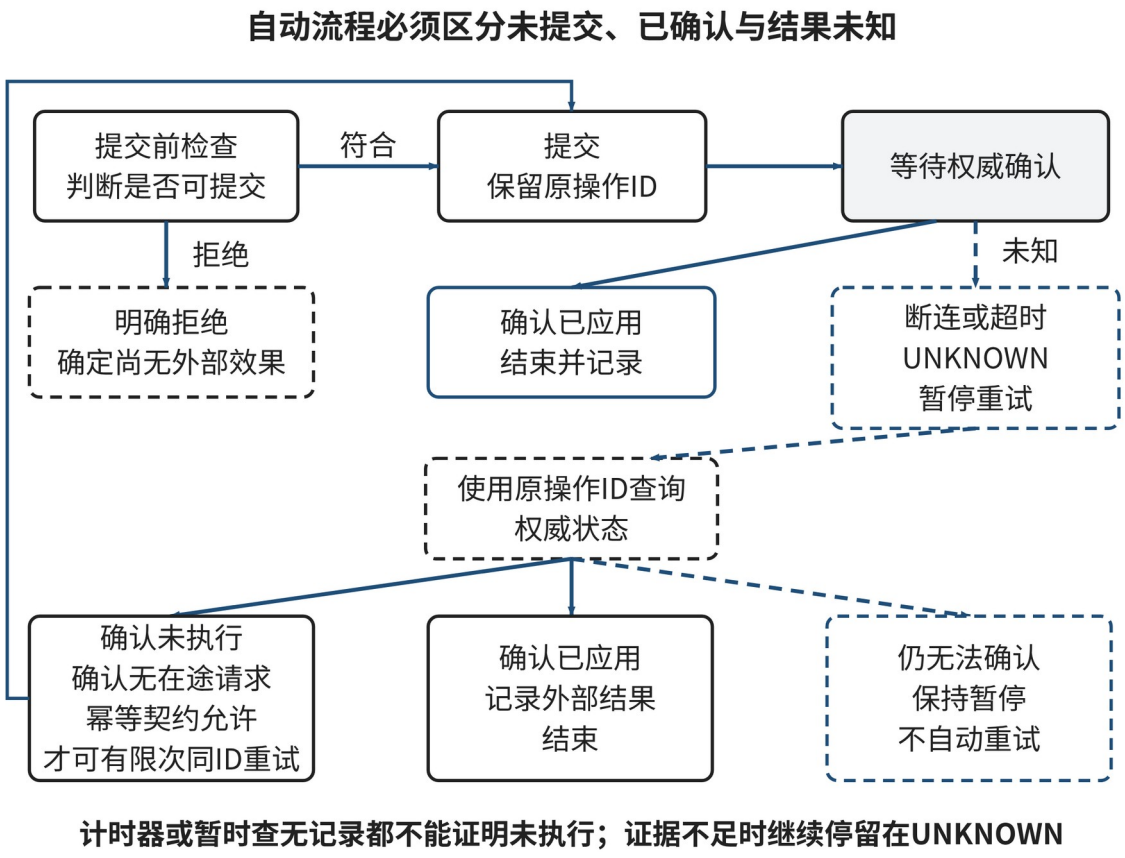


图2-7 自动执行中的检查、确认与未知状态

来源：本书原创教学状态图；提交指可能产生外部效果的教学步骤，不代表已执行任何真实通知。

图 2-7区分提交前的明确拒绝与提交后的结果未知。请沿箭头说明，什么证据允许回到重试，什么情况下应保持暂停。图中的恢复路径要求先取得结果证据，不能由一个计时器自动把未知改成失败。这种状态设计使自动化拥有清晰的停止点，而不只是运行得更久。

2.5.6 反馈要能解释变化来自哪里

闭环不是“得到分数后再让 AI 改一遍”。一次有效反馈应连接输入、结果、失败类型和可修改对象。若路线走错入口，回到表示与地图；若模块分别成功却版本不一致，回到接口与依赖；若新场次预测失准，回到规则的适用范围；若搜索提前

停止漏掉更优候选，回到算法；若分母变化造成假改善，回到评价。相同的“未通过”需要不同修正。

这也解释为什么不能每次失败都换更强的生成模型。生成模型可能帮助提出假设、构造边界测试或解释日志，但它不能替代判断失败属于哪一层。可以给 AI 已核实的失败记录，要求提出互不相同的原因候选和各自需要的检查；不让它直接凭语言合理性选定根因。随后按已有数据或新增观察逐一排除。这是 2.2 的分解与 2.3 的模式方法在反馈中的配合。

人机交互研究提出在系统出错时支持纠正、在长期使用中提供控制等设计原则[10]。在本章任务中，具体表现为让用户看到适用条件、拒绝原因和需要确认的事项，并能修正输入或停止发布。该研究给出设计指导，不能证明本章某个界面已经提升了学生理解或用户满意度。实际效果仍需在相应使用情境中观察。

持续观察可以选择少量具有明确含义的触发条件。例如发现一条已发布路线引用旧地图版本，就暂停受影响结果并重新检查；连续出现本来未覆盖的请求类型，就增加数据核对；某次查询超过限定预算，就返回当前确定的状态而非假装完成。触发值应来自任务容忍度和可恢复方式，不能把随意设的数字称为普遍安全阈值。保留事件的实际时间与输入，使改动后的结果能与改动前对应。

2.5.7 E05：选择方案并设计恢复条件

以下八个请求在同一地图版本与同一组已确认条件下进行离线模拟，记录符合全部通行硬条件的路线及其总分钟数。“失败”表示没有返回合格路线，不是已让

访客走了错误路线。N 组无额外通行要求，A 组要求无台阶。所有数据均为教学示例。

表2-9 E05 固定请求上的路线结果

请求	组别	基线路线分钟	候选路线分钟
U1	N	8	6
U2	N	10	8
U3	N	9	7
U4	N	11	9
U5	A	12	9
U6	A	14	失败
U7	A	失败	失败
U8	A	13	10

来源：本书原创模拟测试数据，不代表现实算法或人群性能。

采用门槛为：A 组至少 3/4 请求成功；每条返回路线都符合全部通行条件；成功路线均不超过 15 分钟。通过门槛后，可参考成功请求均值、全体覆盖和一个辅助综合量作选择。该辅助量将每个失败记为 16 分钟，是题设的评价代价，不是实际出行时间。

请计算两种方法的总体及分组成功数、成功请求均值、按共同成功请求配对的平均差，以及辅助综合量。依据门槛作采用判断，并解释为何不同指标可能支持不同方向。不能把辅助量较低当成自动取消门槛的理由。再给出两个当前表不足以支持的现实结论，以及对应还需哪些材料。

查询计算另有模拟测量：基线每次 40 毫秒；候选预处理 120 毫秒，之后每次 10 毫秒，地图版本不变时可以复用。求不同查询次数下的总成本和严格更快的条件。地图每次都变化时重算，不把行走分钟与计算毫秒相加。

最后处理一个自动执行片段：将路线包发送到展示端，发出后连接断开，未取得确认。只读查询若能按原操作 ID 确认已接收、确认未接收或仍未知，分别怎样处理？若没有查询能力，怎样保存状态与停止？给出一个有限恢复流程和所需契约，不实际发送任何内容。评价看分母、硬门槛、成本口径及未知状态是否一致，参考解见章末 E05。

2.5.8 本章方法如何组成一个可靠过程

评估把前四节的选择变成可比较的证据。表示决定我们计算什么，分解决定信息怎样衔接，模式决定哪些经验能被调用，算法决定怎样得到结果，反馈又帮助定位哪一项需要改变。它们可以往返配合，不必每次按同一顺序从头经过。对一个明确的排序错误，直接检查比较键和稳定性可能足够；对入口未知，则不能靠重跑排序解决。

反思 E05 的采用判断：你是否曾被成功均值吸引而忽略分母？有没有把实现速度与任务效果混成一个数？当确认丢失时，哪条证据真正允许你恢复行动？这些问题要求你解释方法与结果之间的关系。章末综合训练将要求在一张新地图上把这种解释与实际路线、测试和有限 AI 辅助一起交付。

章末综合训练

E06：社区导览的模型、策略与交付证据

一个虚构社区服务区希望给访客提供导览：从入口 G 出发，到服务点 C 领取资料，随后到出口 X。P 为小广场，Q 为展板区，D 为连廊。以下是完整的教学地图，与前文讲解和独立练习不是同一数据。没有列出的道路不存在；所有列出道路均可双向走，代价为确定的分钟数。只有 P—Q 有台阶，其余道路已确认无台阶。

表2-10 E06 模拟参访地图的道路

两端	步行时间／分钟	通行条件
G—P	2	无台阶
G—Q	4	无台阶
P—Q	1	有台阶
P—C	3	无台阶
Q—C	2	无台阶
P—D	5	无台阶
C—D	2	无台阶
Q—X	6	无台阶
D—X	3	无台阶
C—X	7	无台阶

来源：本书原创教学地图，非真实社区设施、开放制度或调查数据。

在 C 领取资料需要 2 分钟，任务只要求领取一次；每次首次到达 C 后立即办理，并在服务完成后继续走。办理必须在出发后第 8 分钟之前或恰于第 8 分钟完成，且必须在第 12 分钟之前或恰于第 12 分钟到 X。所有访客从时刻 0 出发，不需

要其他等待。不要求避免经过同一地点；算法可以通过状态与代价排除无益绕行。分别处理允许台阶与要求无台阶两种 profile，不将它们合并。

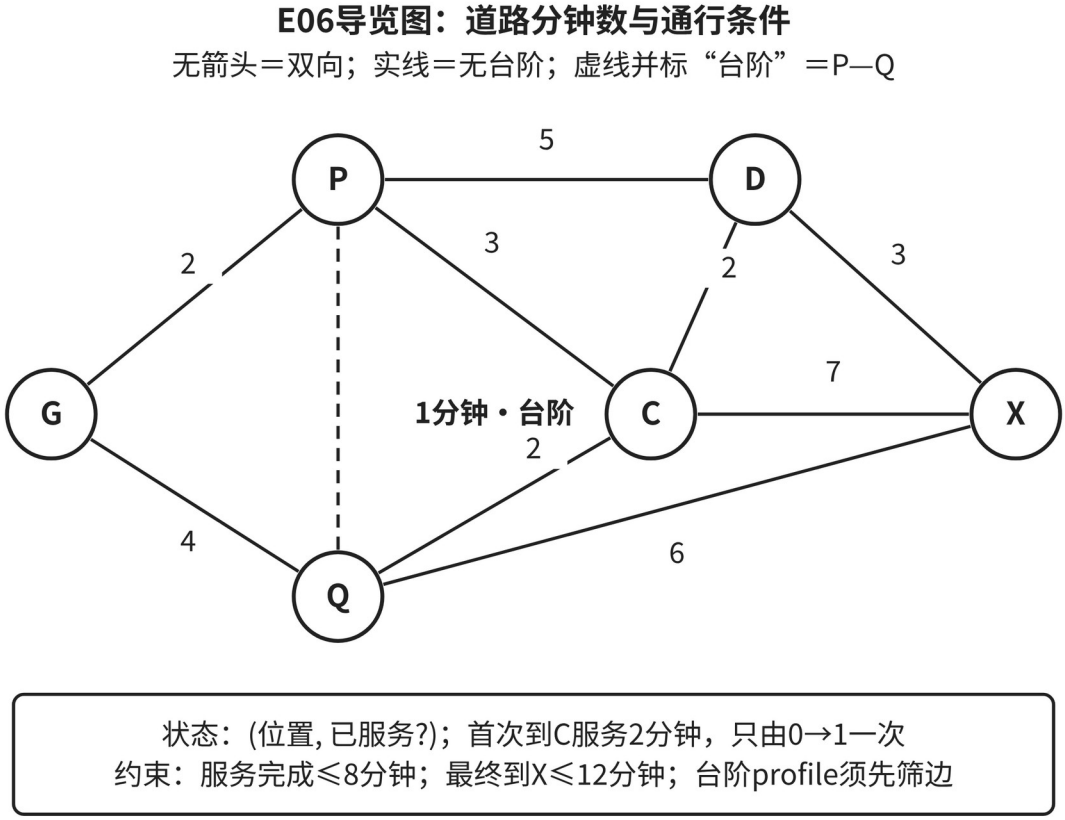


图2-8 E06 社区导览地图

来源：本书原创教学图，根据表 2-10绘制；实线表示无台阶道路，标注“台阶”的虚线表示 P—Q，数字单位为分钟。

阅读图 2-8时将每条线与道路表对应，先找出 profile 筛选会删除哪一条边，再考虑怎样表示“已经领取”。图展示道路关系，不暗示图形距离与步行时间成比例。领取所用 2 分钟属于节点服务代价，不得遗漏或重复加入。

任务一：建立表示与职责。交付节点/道路模型和状态定义，说明为何只保存当前位置不足够。写出通行筛选、领取更新和完成条件；将整项任务分成可检验的职责，说明每个职责的输入、输出、版本与失败结果。地图统一标记为 M1，规则为 C1，各步骤不能使用不一致版本。无需实现网站或接口部署。

任务二：比较两个处理过程。基线在满足 profile 的状态图上，按道路段数优先搜索，邻接按节点字母升序，同段数时保留先发现候选；候选按总分钟数搜索。两者都在首次到 C 时完成领取，并检查服务与到达截止。请先求各自的最少道路段或最短时间候选，再检查硬条件；若某一候选超时，不能声称该 profile 整体无解。给出两种 profile 的路径、逐段累计时间、服务完成时刻与到 X 时刻。可手推，也可用短程序，解释为什么选出的结果对应所定义目标。

任务三：设计并执行边界检查。至少包含：去掉领取条件、改变 profile、关闭 C—D、把某个需要的通行状态改成未知、打乱边表顺序、同一输入重复运行。明确哪些检查应保持结果代价不变，哪些应改变可行集合，哪些只能得到待确认。打乱顺序不要求保留同一条并列最优路径，但应保留最优代价及约定的确定平局规则。已知 C—D 关闭时，其他道路仍按 M1 有效，硬截止不变，重新求解并解释是否有解。

任务四：有限 AI 辅助与评估。任选一项让 AI 提出表示、伪代码或边界测试；只提供上述材料，不允许补造设施。保存候选中的一个具体判断并用手算或程序检查。没有 AI 时，可审查下面的预给候选：“先求 G 到 X 最短路线；如果不经过 C，在结果中插入 C；遇到关闭边，把它设成一个很大的时间。”指出这种候选何时会改变原搜索问题，并给出可验证修正。评价两种方法时分别记录可行率、路线分钟和计算成本；不得把六节点模拟表现推广为真实用户效果。

最终交付一份可复算的小报告，包含模型、依赖图、策略说明、关键状态推演、测试记录和修订依据。评价如下：模型与状态 25%，算法解释与硬条件 25%，

边界测试 25%，来源/模拟边界及有限 AI 核验 15%，表达与复现 10%。这是一份教学量规，尚无课堂评分效度证据。完整参考解在章末 E06。

本章回顾

本章提供了一条更细的检查路径。首先，抽象应保留会改变任务判断的区别；两种情境在表示中相同而结论不同，是有力的修订证据。其次，分解既要减少局部复杂性，也要使接口语义、版本和依赖可以被检查。第三，从样本归纳的规则只在已观察与已检验范围内获得支持，例外帮助定位适用条件。第四，算法的成立依赖状态、代价与不变量，熟悉的算法名不能代替这些条件。最后，评估必须明确比较单位和失败处理，自动执行还要区分已知结果与未知结果。

这些方法共同帮助你解释一个结果为什么可信、为什么失败、怎样修正。遇到问题时不必整套流程重做，也不能总归咎于代码细节或工具不够强。应当沿着具体失败寻找需要改变的对象，并保留能够复核的证据。后续第三章进一步讨论人机协作与 AI 素养，第四章深化证据和论证；它们在这里的计算过程之外加入更多判断条件。

关键术语

任务相关抽象：按照要回答的问题选择对象、关系与性质；简化后仍须检查哪些判断被保留。

状态：足以决定后续合法动作、代价和完成条件的信息摘要，可以包含位置及必要的任务进度。

接口契约：供给方与使用方对输入、输出、语义、版本及失败情形的共同约定。

依赖与关键链：一项工作所需的前置输出，以及在给定并行条件下限制完成时间的依赖链。

规则与例外：对有限观察的概括及不符合该概括的记录；例外可能要求修订范围，也可能反映噪声，须查证。

不变量：处理过程中在关键步骤保持成立的性质，用来解释每次状态变化为何仍支持目标。

稳定排序：键相同的记录保持原有相对顺序；它保证已有次序得到保留，不自动选定合理次序。

评估分母与覆盖：纳入某项指标的比较单位及其中获得合格结果的范围，二者决定平均值能说明什么。

摊销与幂等：前者解释一次准备成本如何由多次处理分担；后者描述重复操作对约定目标状态是否产生与一次相同的效果，两者是不同概念。

参考解与讨论

先独立完成正文练习，再核对以下推演。开放题按证据、推理和适用边界评价；合理的不同表达与设计可以成立。

E01 参考解与讨论

原任务要求无台阶，因而 H→J 与 J→L 都不能进入可行子图。可行路线 H→K→L 共 3＋4＝7 分钟，09:57 出发，10:04 到达，满足 10:05 截止。L→H 是另

一条已声明方向的边，不能据此补造 H→L。表示中保留方向，避免把返程条件偷换成去程条件。

粗表示合并 J、K 后，如果每对点只留最短时间，就会得到 H→P 为 2、P→L 为 2，总计 4 分钟；这两个最小值来自台阶路线，并不满足 profile。即使给 P 整体标成“可无台阶通行”，也会把 K 的属性错误地用于 J 的边。H→J→L 与 H→K→L 是两条不同的具体路径，代价和可行性不同。把它们映成同一条粗路径，便失去当前任务必须区分的信息。

变化一取消无台阶条件，仍不能直接接受 4 分钟路线。H→J 在 09:59 完成，J→L 需 2 分钟，到 10:01 才完成，而该段要求整段通行最晚 10:00 完成，故违反时间条件。H→K→L 仍在 10:04 到达并符合全部条件。这里检验的是完成边的时刻，不是开始进入边的时刻；改变口径会改变题目。两个条件的失败原因应分别说明，不能用一个笼统“不可用”掩盖。

变化二恢复无台阶条件并将 K→L 通行状态设为未知，此时在已确认可用的无台阶子图中，没有得到 H 到 L 的完整路径。不能把未知编码成可用，也不能宣布现实绝无路线。可说明“当前资料不足以确认符合条件的路径”，需要核实 K→L 的无台阶通行、适用时刻及其他已知条件是否仍有效。在确认前的停止本身是合法结果，不应为了交付一条路线补造证据。

合理的字段选择包括明确入口 ID、边起终点、方向、分钟代价、profile 适用性及时间条件，必要时附核验时间和证据。字段多不等于好模型；需要说明各字段怎样改变本题判断。若用建筑级图作展示，可保留它作为视图，但路线处理仍应使用包含入口和条件的模型。

E02 参考解与讨论

依赖边为 A→B、A→C、B→D、C→D、B→E、D→F、E→F、F→G。两种合法拓扑序可以是“A、B、C、D、E、F、G”和“A、C、B、E、D、F、G”。第二种中 E 在 D 之前并不违法，因为 E 只依赖 B；D 等待 B 和 C，F 等待 D 和 E。拓扑序不是唯一的时间表，只要求每条边的起点出现在终点之前。

无限并行时，各环节尽可能在所有前置项完成后立即开始：

计算可统一写成：开始时刻等于所有前置环节完成时刻的最大值；无前置项时取 0；完成时刻再加本环节用时。这里不能把所有用时简单相加，因为 B 与 C 可同时处理；也不能只沿 B 或 C 任意一支往后加，因为 D 必须同时拿到两份输入。

两条序列只验证依赖先后，不指定真实运行时刻；例如序列把 B 排在 C 之前，并不要求系统必须等 B 结束后才启动 C。工期推演采用无限并行假设，所以 B、C 在 A 结束后同时开始。若题目另有机器数量、人员互斥或阶段间等待条件，就必须把这些约束加入模型，不能继续直接套用此时间表。

表2-11 E02 各环节的最早开始与完成

环节	开始（分钟）	完成（分钟）
A	0	2
B	2	6
C	2	5
D	6	11
E	6	8
F	11	14
G	14	15

来源：本书原创教学材料与作者推演；模拟数值的条件见对应题设。

D 等 B、C 两支，实际由较晚完成的 B 控制；F 等 D、E 两支，实际由较晚完成的 D 控制。因此一条关键链是 A—B—D—F—G，总时长为 2+4+5+3+1=15 分钟。C 和 E 仍是必须完成的环节，但它们在本组时长下不是决定项目终点的分支。

B 给出 R7 节点、C 给出 R8 规则时，D 不能仅因两份资料都含 map_version 与 profile 字段就直接组合。R7 和 R8 明确不一致，意味着资料依据可能不是同一张快照。应核对两个字段的实际值及其发布方定义，并确认 profile 在两端指向同一类规则；路线生成后还要保留相同的版本信息。除非存在已确认的跨版本映射和兼容规则，否则应停止合并并请求统一版本的数据。

若 D 改为 2 分钟，D 仍在 B、C 都完成后于第 6 分钟开始，于第 8 分钟完成；E 也在第 8 分钟完成。F 于是第 8 分钟开始，第 11 分钟完成，G 在第 11 至 12 分钟完成。新关键链有两条：A—B—D—F—G 和 A—B—E—F—G，每条均为 12 分钟。B 之后的 D、E 两支现在同一时刻完成，因此两支都决定 F 能否启动；A—C—D 这一路仍早于 B 支完成。此次 D 减少 3 分钟，工期也从 15 降至 12 分钟，减少 3 分钟；但 D、E 现在并列控制 F，再单独缩短 D 将无法继续缩短工期。例如 D 再降至 1 分钟，它虽在第 7 分钟完成，F 仍须等 E 到第 8 分钟，交付时刻仍为 12 分钟。

评分应接受所有满足依赖边的不同序列；时间表须逐节点用“前置完成时刻最大值”计算；关键链须与总工期一致。版本诊断要依据值和语义兼容性，不能只核对列名，也不能把未确认映射当作已兼容。

重算后，C 完成于第 5 分钟，比 B 晚完成前的依赖等待并未成为 D 的限制；在 D 缩短后，C 仍于第 5 分钟完成，D 仍要等 B 到第 6 分钟，所以工期没有再因 C 变化。第二条关键链经过 E，是因为 B 完成后 E 需 2 分钟，恰好与缩短后的 D 同在第

8 分钟结束。关键链可能不止一条，且改动一个环节后，控制项目终点的路径可能改变。若学生只报告“新工期减少 3 分钟”，却没有重新比较 D、E 两支，就没有说明 F 为何能在第 8 分钟启动。

“关键”在这里表示延迟该链上的活动会延迟最终交付，而不是说其他分支不重要。C 即使不是关键链的一部分，若它延迟超过 B 支的空余时间，仍可能转而控制 D 的开工。因此关键链结论严格依赖给定的确定时长、依赖关系和并行假设；条件变化时需要从依赖图重新计算，而不能把原来的链当成永久属性。

E03 参考解与讨论

先按预测规则逐行分类，开发集 d1—d8 中，RA 预测正类的记录是 d1、d2、d3、d6；RB 还把 d7 判为正类。两条规则都把 d4、d5、d8 判为负类，且这些记录的实际标签均为 0。计数如下：

表2-12 E03 规则在开发集与测试集上的指标

数据范围	规则	TP	FN	FP	TN	准确率	精确率	召回率
开发集	RA	4	1	0	3	7/8	4/4	4/5
开发集	RB	5	0	0	3	8/8	5/5	5/5
测试集	RA	1	1	0	2	3/4	1/1	1/2
测试集	RB	1	1	1	1	2/4	1/2	1/2

来源：本书原创教学材料与作者推演；模拟数值的条件见对应题设。

例如开发集 RA 的准确率是(TP+TN)/8=(4+3)/8，精确率是 TP/(TP+FP)=4/(4+0)，召回率是 TP/(TP+FN)=4/(4+1)。测试集 RB 的精确率为 1/(1+1)，因为 d9

是真正例、d11 是假正例；召回率为 1/(1+1)，因为 d10 是假负例。所有分母均对应当前划分的实际记录，不能把开发集和测试集的计数拼在一起计算。

开发集上，若选择依据是两条给定规则中的召回率，RB 优于 RA；在此数据中 RB 也未增加开发集误报。但测试集只包含四条记录：RA 把 d9 判正，d10 漏判，d11、d12 判负；RB 同样漏判 d10，却把实际未拥堵的 d11 误判为正，且正确判负 d12。两者在测试集的召回率相同，准确率和精确率则不同。开发集表现较好的规则没有在这四条测试记录上全面保持优势。

这些结果只能描述本组模拟数据：RB 增加活动条件后，既覆盖了开发集中的 d7，也在测试集的 d11 产生误报。它们不能证明活动通常导致拥堵，也不能证明 RA 或 RB 在其他时段、入口或人群中更可靠。四条测试记录过少，不能支持显著性或总体性能结论；若在查看这些结果后改写 RB 条件，d9—d12 就已被用于开发，不能再称为独立测试。

若还需要比较改写后的候选，应先明确这四条测试记录已被观察过，随后用新的、未参与决策的独立场次作下一轮评估。新增样本最好覆盖预计使用的时段与入口，并保留同一场次内的记录关系，避免把“更多行”误当成“更多独立证据”。

一个直接反例是 d11：event=1、x=38，但 y=0，它满足 RB 新增的活动条件而实际未拥堵；d7 具备相同的 event=1 与 x>=35 条件，却是拥堵。二者说明，在规则判断相同的不同场次，实际结果仍可能相反；新增条件未能区分这些情况。可提出的新增信息方案之一，是在预测时点记录入口当前开放通道数或可用通行能力，并注明测量时间、计算口径和来源；之后可比较同样人数在不同能力下是否表现不同。它必须在预测前取得，且需要新数据验证，不能预先认定一定提升预测。

对象泄漏反例：同一场次内按分钟记录多行，如果把第 1—5 分钟放入开发集、第 6—10 分钟放入测试集，两个集合共享同一场次的布置、人员和环境。模型可能认出场次特征，误把重复观察当成对新场次的成功预测。应以整场为单位分组后切分；本题给定每条记录来自不同场次，正是为了降低这种泄漏风险。

时间泄漏反例：若在预测时点前五分钟，输入表中却包含“未来十分钟的最高排队人数”或“整场结束后汇总的拥堵时长”，字段已经包含目标时段的信息。即使 d1—d8 早于 d9—d12，也不能使这种字段变成合法输入。应按每条记录的预测时刻核对字段生成时间；未来记录只能用于测试，不能反向参与开发期的归一化或规则选择。

E04 参考解与讨论

三种搜索策略。 BFS 按字母顺序从 S 发现 A、B，再由 A 发现 C、T；T 的前驱因此为 A。队列出队顺序为 S、A、B、C、T，路线是 S—A—T，用时 1+9=10 分钟。它满足最少边数：两段；但边权不等时，不能据此称为最短时间。

当前位置局部贪心从 S 选 S→A（1），到 A 后在 A→B（2）、A→C（1）、A→T（9）中选 A→C，再由 C 到 T。路线 S—A—C—T，用时 1+1+9=11 分钟。它每步都选当前最短可用边，却忽略了从 A 走到 B 再到 T 的累计代价。

Dijkstra 的状态变化如下（未列出的距离为∞）：

表2-13 E04 的 Dijkstra 逐轮距离记录

确定状态	更新	暂定距离（A，B，C，T）
初始化	—	（∞，∞，∞，∞）
S=0	A=1，B=4	（1，4，∞，∞）

确定状态	更新	暂定距离（A，B，C，T）
A=1	B 由 4 改为 3；C=2，T=10	（1，3，2，10）
C=2	T 候选为 11，不优于 10	（1，3，2，10）
B=3	T 由 10 改为 5	（1，3，2，5）
T=5	达到目标	—

来源：本书原创教学材料与作者推演；模拟数值的条件见对应题设。

前驱关系为 A←S、B←A、T←B，返回 S—A—B—T，用时 1+2+2=5 分钟。确定次序是 S、A、C、B、T；C 在 B 前确定，因为 2 小于 3。它与 BFS 的 10 分钟和局部贪心的 11 分钟不同，差别来自算法目标与候选排序规则。

这里目标 T 在处理 A 时就首次被发现，暂定距离为 10；此时 B=3、C=2 仍比 T 小，算法必须继续处理它们，才能把 T 改为 5。若把“发现目标”误当成“最短路线已确定”，就会过早停止并返回错误的 10 分钟。相反，Dijkstra 在 T 以 5 成为当前最小暂定距离时才可以结束。BFS 按本题指定的字母顺序得到 S—A—T；若题目没有固定相邻点顺序，另一条同为两边的路线可能先被发现，但步数结论不变。

可用路线核算复查输出：S—B—T 为 4+2=6，S—A—T 为 1+9=10，S—A—C—T 为 1+1+9=11，而 S—A—B—T 为 5。BFS 选两边的路线，符合最少边数；贪心逐边作局部选择，走到 C 后才发现剩下的路较长；Dijkstra 比较累计距离，把从 A 发现的 B=3 与 C=2 都保留为候选。结果不同不表示只要路线段数多就更优，必须回到任务给定的时间代价。

负边变体。把 B→T 改为−3 后，标准 Dijkstra 的非负边前提被破坏；因此它不再保证所有图都能得到正确答案。但此处按原规则运行仍会得到 S—A—B—T：确定 S 后 A=1、B=4；确定 A 后 B=3、C=2、T=10；确定 C 后 T 仍为 10；确定 B 后 T 更

新为 0；随后确定 T，路径代价为 1+2−3=0。该结果确实是本图改动后的最短简单路线，所以不能把“前提失效”说成“本例必错”。算法适用条件说明普遍保证；单例输出是否正确要另行核算。

本例碰巧有效，是因为潜在的负边终点 T 在距离 0 时才被确定；没有一个尚未处理、能经负边产生更短路线的状态留在前沿。最小反例中则有 B=5 尚未处理，且 B→A=−4 会把已确定的 A 从 2 降为 1。差别在于负边是否能从后处理状态回头改善已确定距离，而不是负数符号本身必然制造错误。

最小反例可用三个顶点 S、A、B 及边 S→A=2、S→B=5、B→A=−4。若 A 是目标，Dijkstra 会在确定 A=2 时提前返回；若继续求所有点距离，随后处理 B=5 也不会修改已确定的 A。但路线 S→B→A 代价 5−4=1，短于 2。在不含自环且要求不存在负权环的有向简单图中，至少需要起点、先确定的状态和稍后处理的状态；直接路线与两段替代路线至少需要这三条边，因此该例在顶点数和边数上都达到下界。

稳定排序。 按 submitted 升序作第一轮稳定排序，次序为 r3、r4、r2、r1。再按 priority 升序稳定排序后为 r4、r2、r3、r1；priority 为 1 的 r4、r2 按 submitted 升序排列，priority 为 2 的 r3、r1 也按 submitted 升序排列。一次按（priority 升序，submitted 升序）比较的复合键得到相同次序。稳定排序保留 priority 和 submitted 都相同的记录在上一轮中的相对次序；本题数据没有双键全同的记录。若需要登记编号等新的平局规则，应明确加入排序键，不能把偶然的原始顺序解释成业务偏好。

复核最终顺序时，先确认 priority=1 的 r4（09:01）早于 r2（09:02），再确认 priority=2 的 r3（09:01）早于 r1（09:03）。若两轮都用不稳定排序，第二轮可能打乱第一轮建立的同 priority 次序，输出便不再可靠地实现原定的复合规则。

E05 参考解与讨论

基线 N 组 4/4 成功，A 组 3/4 成功，总体 7/8；候选 N 组 4/4，A 组 2/4，总体 6/8。基线成功时间合计 77 分钟，均值 11 分钟；候选合计 49 分钟，均值 49/6，约 8.17 分钟。两种成功均值的分母不同，所以不能仅凭这两个数作整体采用判断。

共同成功的是 U1、U2、U3、U4、U5、U8。基线在这些请求上合计 63 分钟，候选 49 分钟，配对平均差为（63－49）/6＝7/3，约 2.33 分钟，表明在这六个共同成功请求上候选路线平均更短。U6 只在基线成功，U7 两种都失败，应另列，而不能在配对结论中消失。

按失败赋 16 分钟的辅助量，基线为（77＋16）/8＝11.625，候选为（49＋32）/8＝10.125 分钟。候选辅助量较低，但它在 A 组只成功 2/4，未达至少 3/4 的硬门槛。基线返回时间均不超过 15，且题设声明成功均符合通行条件，满足给定采用门槛。因此当前应保留基线，同时调查 U7；候选可以继续开发，不能以辅助量抵消门槛。

分组成功均值也有助于解释：基线 N 组 38/4＝9.5，A 组 39/3＝13；候选 N 组 30/4＝7.5，A 组 19/2＝9.5。A 组的较短均值同样排除了失败请求。无论看总体还是分组，都须同时保留覆盖信息。题目没有给重复运行的分布、用户体验、现场拥堵和长期维护记录，不能宣称统计显著改善、现实平均节约或对所有访客更优。

计算查询成本时，基线为 40q 毫秒，候选为 120＋10q 毫秒。严格更快条件是 30q＞120，即 q＞4；q＝4 相同。q＝1 时分别 40 与 130，q＝5 时 200 与 170。若每次地图变化都要重做预处理，候选为 130q，反而高于 40q。该结果仅对应给定计算测量，不能与路线分钟直接相加。

发送后的确认丢失保留结果未知和原操作 ID。只读查询确认已接收时记录成功，不再重复发送；确认原动作确实未产生效果时，只有该查询具有足以排除在途提交的语义、当前版本和权限仍有效，才可按约定有限次数恢复提交；仅仅暂时没查到记录不一定等于未接收。若查询仍未知，保持暂停并交由确定的恢复处理；没有查询能力时同样保存未知，不盲目重发。

如果接收方提供幂等契约，可用相同操作 ID 识别同一次动作，但须核对去重范围、并发与保留期。新 ID 不是对原动作的安全重试。开放回答可以有不同实现，评价要求是不能把无确认当成明确失败、不能在缺少恢复依据时产生额外效果，并记录停止条件与所缺证据。

E06 参考解与讨论

可用状态由位置及是否已领取组成，累计分钟数用于比较和截止检查；图、规则及 profile 是固定输入。首次到 C 加 2 分钟并完成领取，其他状态转移不重复加入服务时间。只有已领取状态下到 X 才是目标；服务完成时间须不超过 8，目标时间不超过 12。各模块需要交付相同 M1/C1/profile，不能只提供一段缺少这些语义的地点列表。

按道路段数的基线找到 G→P→C→X，共三段道路，走路 2＋3＋7＝12，加领取 2 后 14 分钟。服务在第 7 分钟完成，符合服务截止，但第 14 分钟到 X 违反总截

止。G→Q→C→X 也为三段但更晚，字母顺序规则先发现前者。允许台阶或无台阶 profile 均可以得到 G→P→C→X；基线候选失败不能证明任务无解。

按总时间搜索，无台阶 profile 的一条最优路线是 G→P→C→D→X：到 P 为 2，到 C 为 5，领取后 7，到 D 为 9，到 X 为 12。两项截止都满足。允许台阶时 G→P→Q→C→D→X 也为 12：到 P2、Q3、C5，领取后 7、D9、X12。因此允许台阶并不在本图必然缩短最优时间，它只是扩大可行集合；两个并列最优路线不同，不构成算法错误。

用可分解的下界核对最优性：G 到 C 最短步行 5 分钟，两种 profile 都能实现；C 到 X 最短为 C→D→X，共 5 分钟；领取 2 分钟，合计至少 12，并有完整路线达到该值。服务截止在达到 C 后完成时检查。本图静态非负，且必须先经过 C、服务一次后到 X，这个分段论证成立；若边权依赖到达时间、领取造成路径许可变化或存在其他顺序条件，不能自动沿用同样分段。

去掉领取条件时，无台阶最短可取 G→P→D→X，为 10 分钟；允许台阶可取 G→P→Q→C→D→X，也是 10 分钟。这些路线不再需要加 2 分钟，也不必到 C。输入变化确实改变完成条件，应重算，不能让旧缓存省略该区别。

仅关闭 C—D 后，G 到 C 仍需至少 5 分钟，C 到 X 的最短可用走法为直接 C→X，7 分钟；其他候选 C→Q→X 需 8 分钟，C→P→D→X 需 11 分钟。总时间下界 5＋2＋7＝14，存在 G→P→C→X 达到 14，因此在当前第 12 分钟总截止下无解。延长截止或修建新路只能作为改变条件的提案，不能混入当前可行解。若 C→X 的通行状态同时未知，仍不能把未知设成已确认可用；只使用已知可用边时，最短总时间为 15 分钟。还可以作乐观下界检查：保持题设中的 7 分钟代价及其他条件不

变，即使暂按这条边可用来扩大候选集合，最短总时间也为 14 分钟，仍超过 12 分钟。因此在本题固定模型与截止条件下，可以确定无可行解；边是否通行仍待核实。这是下界排除，不是将未知边确认为可用，也不代表掌握了完整现实道路信息。

打乱输入顺序时，精确算法最优代价应保持 12；若承诺字母平局规则，应先统一邻接顺序再搜索。重复同一输入也应给出相同约定结果。表示以确认子图、任务标记和分钟代价为基础，检查可以针对非法方向、遗漏服务、重复服务、超时、旧版本与未知边。这些失败比只测“有没有一条路径”更能区分处理是否正确。

预给候选先求 G 到 X 再插入 C，会在插入后改变路径连接与时间，原最优性不再成立；插入还可能漏加服务。把关闭边设成大时间也不等于禁止，若没有其他路，程序仍可能选它。修正为先筛掉明确不允许的边，在已定义状态上搜索并检查截止；未知边另列，不伪装成很慢的可用边。

开放报告允许不同代码、图形和分解边界。参考结构可分为输入核对、 profile 筛选、状态转移、搜索、独立结果检查和输出说明；每项的失败应能回到明确输入或规则。AI 产物与手工候选都必须经过同样检查。六节点数据支持的是本模型中的可复算结果，尚不能支持实际社区无障碍、拥堵预测、用户体验或教学效果结论。

参考文献

[1] Wing J M. Computational Thinking: What and Why?[EB/OL]. (2010-11-17)[2026-10-11]. https://www.cs.cmu.edu/~CompThink/resources/TheLinkWing.pdf.

[2] Parnas D L. On the Criteria To Be Used in Decomposing Systems into Modules[J/OL]. Communications of the ACM, 1972, 15(12): 1053-1058[2026-10-11]. https://doi.org/10.1145/361598.361623.

[3] scikit-learn developers. Cross-validation: evaluating estimator performance[EB/OL]. [2026-10-11]. https://scikit-learn.org/stable/modules/cross_validation.html.

[4] scikit-learn developers. Common pitfalls and recommended practices[EB/OL]. [2026-10-11]. https://scikit-learn.org/stable/common_pitfalls.html.

[5] MIT OpenCourseWare. 6.006 Introduction to Algorithms: Lecture Notes[EB/OL]. 2011[2026-10-11]. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/pages/lecture-notes/.

[6] Dijkstra E W. A note on two problems in connexion with graphs[J/OL]. Numerische Mathematik, 1959, 1: 269-271[2026-10-11]. https://doi.org/10.1007/BF01386390.

[7] Python Software Foundation. Sorting Techniques[EB/OL]. [2026-10-11]. https://docs.python.org/3/howto/sorting.html.

[8] Tabassi E. Artificial Intelligence Risk Management Framework (AI RMF 1.0): NIST AI 100-1[R/OL]. 2023-01[2026-10-11]. https://doi.org/10.6028/NIST.AI.100-1.

[9] Fielding R, Nottingham M, Reschke J. HTTP Semantics: RFC 9110[S/OL]. 2022-06[2026-10-11]. https://www.rfc-editor.org/rfc/rfc9110.html#section-9.2.2.

[10] Amershi S, Weld D, Vorvoreanu M, et al. Guidelines for Human-AI Interaction[C/OL]//Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems. New York: ACM, 2019: 1-13[2026-10-11]. https://doi.org/10.1145/3290605.3300233.